



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Análisis y Visualización de Datos
Masivos/ Visual Analytics and Big Data

Diseño e implementación de una arquitectura de datos masivos para el análisis predictivo y la visualización interactiva

Trabajo fin de estudio presentado por:	Laura Carrasco Mateos
Tipo de trabajo:	Desarrollo de software
Director/a:	Dr. Luis Miguel Fernández-Galván
Fecha:	Septiembre de 2026

Resumen

Este Trabajo Fin de Máster tiene como objetivo diseñar e implementar una arquitectura modular de datos orientada al análisis predictivo y a la visualización interactiva. Para validar la propuesta se ha desarrollado un caso práctico basado en datos abiertos de tráfico urbano de la ciudad de Madrid, utilizando información histórica correspondiente a los años 2023 y 2024 y datos de ubicación de los puntos de medida publicados por el Ayuntamiento de Madrid. La arquitectura implementada integra las fases de ingesta, limpieza, transformación, construcción de un dataset analítico, modelado predictivo y representación visual mediante un informe interactivo en Power BI.

El proceso de ingesta se ha aplicado sobre 24 ficheros mensuales en formato CSV, con un volumen inicial de 308.458.911 registros. Para construir un prototipo funcional en un entorno local se seleccionaron 50 puntos de medida. Tras aplicar los procesos de filtrado, conversión de tipos, eliminación de registros erróneos y agregación horaria, se obtuvo un dataset final de 833.086 registros horarios completos. Posteriormente, se formuló un problema de regresión supervisada para estimar la intensidad horaria de cada punto a partir de información temporal e histórica disponible anteriormente. El modelo Random Forest Regressor se evaluó mediante una división temporal de los datos y se comparó con dos modelos de referencia basados en la intensidad de la hora anterior y en la misma hora del día anterior.

Los resultados obtenidos muestran que Random Forest supera a ambos modelos de referencia, alcanzando un MAE de 52,48 vehículos/hora, un RMSE de 89,29 vehículos/hora y un R^2 de 0,9565 sobre el conjunto de prueba de 2024. Finalmente, los datos procesados, las predicciones, las métricas y los indicadores de calidad se integraron en un informe interactivo de ocho páginas en Power BI. La solución desarrollada permite integrar datos abiertos, procesamiento analítico, aprendizaje automático y visualización interactiva dentro de una arquitectura modular implementada en un entorno local, cuya ampliación a otros escenarios requeriría validaciones adicionales.

Palabras clave: análisis predictivo, movilidad urbana, Power BI, Random Forest, arquitectura de datos.

Abstract

This Master's Thesis aims to design and implement a modular architecture focused on predictive analytics and interactive visualization. To validate the proposal, a practical case study was developed using open urban traffic data from the city of Madrid, using historical information from 2023 and 2024 together with location data for measurement points published by the Madrid City Council. The implemented architecture integrates the stages of ingestion, cleaning, transformation, analytical dataset construction, predictive modeling and visual presentation through an interactive Power BI report.

The ingestion process was applied to 24 monthly CSV files, with an initial volume of 308,458,911 records. To build a functional prototype in a local environment, 50 measurement points were selected. After applying filtering, data type conversion, error removal and hourly aggregation processes, a final dataset of 833,086 complete hourly records was obtained. A supervised regression problem was then formulated to estimate hourly traffic intensity at each point based on the previously available temporal and historical information. The Random Forest Regressor model was evaluated using a temporal data split and compared against two baseline models, one based on the previous hour's intensity and another on the intensity at the same hour on the previous day.

The results show that Random Forest outperforms both baseline models, achieving an MAE of 52.48 vehicles/hour, an RMSE of 89.29 vehicles/hour and an R^2 of 0.9565 on the 2024 test set. Finally, the processed data, predictions, metrics and quality indicators were integrated into an eight-page interactive Power BI report. The developed solution demonstrates the feasibility of integrating open data, analytical processing, machine learning and interactive visualization within a modular architecture implemented in a local environment, whose extension to other scenarios would require further validation.

Keywords: predictive analysis, urban mobility, Power BI, Random Forest, data architecture.

Índice de contenidos

1.	Introducción	1
1.1.	Motivación	3
1.2.	Planteamiento del trabajo	4
1.3.	Estructura del trabajo	6
2.	Contexto y estado del arte	8
2.1.	Contexto del problema	8
2.2.	Estado del arte	9
2.2.1.	Arquitecturas Big Data para el análisis de datos masivos	9
2.2.2.	Análisis predictivo aplicado a la movilidad urbana	12
2.2.3.	Visualización interactiva de datos de movilidad	14
2.2.4.	Herramientas y soluciones existentes	15
2.3.	Conclusiones	17
3.	Objetivos concretos y metodología de trabajo	19
3.1.	Objetivo general	19
3.2.	Objetivos específicos	19
3.3.	Metodología del trabajo	21
3.3.1.	Enfoque metodológico	21
3.3.2.	Fases del desarrollo	22
3.3.3.	Trazabilidad y control de la evaluación	25
3.4.	Uso de herramientas de inteligencia artificial	26
4.	Marco normativo	27
4.1.	Protección de datos personales	27
4.2.	Reutilización de información del sector público	28
4.3.	Normativa relacionada con movilidad	29

4.4.	Consideraciones éticas y de seguridad.....	30
4.5.	Aplicación al TFM.....	31
5.	Desarrollo específico de la contribución.....	32
5.1.	Descripción general de la arquitectura propuesta.....	32
5.2.	Requisitos de la solución	34
5.2.1.	Requisitos funcionales.....	34
5.2.2.	Requisitos no funcionales.....	36
5.3.	Instanciación de la arquitectura en el caso de la movilidad urbana.....	37
5.4.	Tecnologías seleccionadas y justificación.....	39
5.5.	Fuentes de datos utilizadas	41
5.6.	Proceso de ingesta y caracterización de los datos	42
5.7.	Limpieza, transformación y agregación horaria.....	43
5.8.	Construcción del dataset analítico	45
5.9.	Modelado predictivo de intensidad del tráfico.....	47
5.10.	Estrategia de entrenamiento y evaluación	48
5.11.	Importancia de las variables.....	49
5.12.	Análisis complementario de los errores.....	50
5.13.	Diseño del informe interactivo en Power BI	51
5.14.	Validación de la solución	52
5.14.1.	Pruebas funcionales	52
5.14.2.	Prueba de aceptación y usabilidad.....	58
5.14.3.	Resultados de la validación	59
5.15.	Reutilización y posibilidades de ampliación	60
6.	Código fuente y datos analizados	62
6.1.	Código fuente y organización del repositorio	62

6.1.1.	Scripts desarrollados	63
6.1.2.	Documentación y dependencias	64
6.2.	Datos analizados	65
6.2.1.	Datos originales utilizados	65
6.2.2.	Datos procesados generados	65
6.2.3.	Archivos destinados a Power BI	67
6.3.	Disponibilidad del código y los datos	68
7.	Conclusiones.....	69
7.1.	Conclusiones del proyecto	69
7.2.	Relación entre objetivos y resultados obtenidos	70
7.3.	Conclusiones personales y aprendizaje adquirido	72
8.	Limitaciones y prospectiva	74
8.1.	Limitaciones.....	74
8.2.	Trabajo futuro.....	75
8.3.	Prospectiva de la arquitectura	77
	Referencias bibliográficas.....	79
Anexo A.	Privacidad y protección de datos	83
Anexo B.	Script de ingesta y limpieza	87
Anexo C.	Script de modelado predictivo	93
Anexo D.	Script de exportación a Power BI	98
Anexo E.	Imágenes del Dashboard en Power BI.....	101
Anexo F.	Declaración de uso de herramientas de IA	105

Índice de figuras

Figura 1. Arquitectura general propuesta para el análisis predictivo y la visualización interactiva.

Figura 2. Arquitectura aplicada al caso práctico de predicción de intensidad del tráfico urbano de Madrid.

Figura 3. Organización del repositorio donde se encuentra el proyecto

Índice de tablas

Tabla 1. Diferencias entre los distintos enfoques.

Tabla 2. Objetivos específicos.

Tabla 3. Componentes principales de la arquitectura implementada.

Tabla 4. Requisitos funcionales de la solución

Tabla 5. Requisitos no funcionales de la solución

Tabla 6. Tecnologías utilizadas en la implementación

Tabla 7. Fuentes de datos utilizadas en el caso de estudio

Tabla 8. Resumen cuantitativo del proceso de ingesta y preparación.

Tabla 9. Principales controles y transformaciones aplicados durante la preparación de los datos

Tabla 10. Principales grupos de variables utilizados en la preparación del conjunto analítico

Tabla 11. Métricas de evaluación del modelo predictivo.

Tabla 12. Variables con mayor importancia en el modelo Random Forest.

Tabla 13. Validación de la solución.

Tabla 14. Validación de la solución.

Tabla 16. Scripts desarrollados durante el proyecto

Tabla 17. Principales archivos procesados generados por la solución.

Tabla 18. Relación entre los objetivos específicos y los resultados obtenidos.

1. Introducción

La generación y la disponibilidad de grandes volúmenes de datos han transformado la forma en que las organizaciones, administraciones públicas y otras entidades pueden estudiar fenómenos complejos y fundamentar sus decisiones. El interés de los datos masivos no se encuentra solamente en su volumen, sino también en la posibilidad de combinar fuentes heterogéneas, aplicar técnicas de procesamiento y de análisis para obtener el conocimiento que difícilmente sería accesible mediante procedimientos tradicionales. Es por eso por lo que las tecnologías de Big Data y la analítica avanzada han impulsado nuevas formas de investigación y explotación de la información, aunque su utilidad depende de la calidad de los datos que se tengan, de las decisiones metodológicas y del correcto uso de las herramientas en función al problema que se quiere analizar (Kitchin, 2014).

Sin embargo, la disponibilidad de los datos no garantiza por sí misma que puedan utilizarse de forma adecuada. Los conjuntos de información pueden encontrarse distribuidos en diferentes archivos o plataformas, presentar estructuras y formatos diversos, contener registros incompletos o inconsistentes y necesitar transformaciones específicas antes de su análisis. Por ello, es necesario organizar procesos que permitan obtener, depurar, estructurar y relacionar toda esa información, manteniendo la trazabilidad de las operaciones realizadas. La integración de estas fases dentro de un flujo de trabajo coherente permite construir conjuntos de datos preparados para su posterior utilización en modelos predictivos y herramientas de visualización.

Este Trabajo de Fin de Máster, titulado *Diseño e implementación de una arquitectura de datos masivos para el análisis predictivo y la visualización interactiva*, se centra en esta necesidad mediante el desarrollo de una solución tecnológica modular que integra las principales fases del tratamiento de los datos. La solución se ha implementado en un entorno local utilizando Python y pandas para la ingesta y la transformación de los datos, archivos estructurados como soporte de intercambio y almacenamiento, scikit-learn para el modelo predictivo y Power BI para la visualización interactiva. El diseño muestra una separación funcional entre sus componentes, de manera que cada uno desempeña una responsabilidad concreta y genera resultados que pueden ser utilizados por las fases posteriores.

Para comprobar el funcionamiento de la solución se ha desarrollado un caso práctico centrado en el tráfico urbano de Madrid. El Ayuntamiento de Madrid pone a disposición pública datos históricos de sus puntos de medida, junto con información complementaria sobre su ubicación y características. Estos datos permiten estudiar la evolución temporal y espacial del tráfico a partir de mediciones registradas por la infraestructura municipal. Su utilización proporciona un escenario real para implementar y evaluar el flujo de procesamiento y análisis desarrollado en este trabajo.

El caso de estudio utiliza 24 archivos mensuales correspondientes al periodo comprendido entre enero de 2023 y diciembre de 2024, además de los datos geográficos asociados a los puntos de medida. El conjunto de partida contiene 308.458.911 registros. Sobre esos datos se aplicaron los procesos de selección, limpieza y agregación definidos en el pipeline para 50 puntos de medida, obteniéndose finalmente 833.086 observaciones horarias completas. Este conjunto constituye la base del análisis y del modelado predictivo desarrollado posteriormente. Estas cifras describen el alcance de los datos procesados y, por tanto, los resultados obtenidos no pretenden caracterizar de forma exhaustiva de toda la red viaria de Madrid.

La finalidad del trabajo comprende tanto la arquitectura de procesamiento como los componentes necesarios para llevarla a la práctica. Incluye los procesos de ingesta y preparación de los datos, la construcción del conjunto analítico, el entrenamiento y la evaluación de un modelo predictivo y la generación de las tablas utilizadas posteriormente en Power BI. Para el componente predictivo se ha utilizado Random Forest Regressor, un método basado en la combinación de árboles de decisión que permite abordar problemas de regresión (Breiman, 2001). El problema se ha formulado para estimar la intensidad de tráfico de una hora objetivo utilizando información temporal e histórica disponible previamente, evitando incorporar variables contemporáneas que no estarían disponibles en el momento de realizar la predicción.

La evaluación final se ha realizado mediante una separación temporal de los datos de entrenamiento y los de prueba. El Random Forest se compara con dos modelos de referencia: uno basado en la intensidad registrada durante la hora anterior y otro basado en la intensidad correspondiente a la misma hora del día anterior. Este procedimiento permite valorar el comportamiento del modelo frente a predicciones temporales sencillas y mantener el orden

cronológico de las observaciones durante la evaluación. El análisis detallado de las métricas, la cobertura de las predicciones y la distribución de los errores se presenta en el capítulo dedicado al desarrollo y evaluación de la solución.

La arquitectura se ha planteado siguiendo los criterios de modularidad y separación de responsabilidades para facilitar el mantenimiento de sus componentes y su posible adaptación. No obstante, la validación realizada se limita al caso de estudio seleccionado y a una ejecución en un entorno local. La escalabilidad mediante procesamiento distribuido, la aplicación efectiva de la arquitectura a otros dominios y su despliegue en un entorno de producción quedan fuera del alcance de la implementación evaluada y requerirían pruebas específicas.

1.1. Motivación

La motivación principal de este Trabajo Fin de Máster surge del interés por desarrollar una solución que integre las distintas etapas del tratamiento de los datos, desde su obtención y preparación hasta el análisis predictivo y la visualización de los resultados. La disponibilidad de grandes volúmenes de información amplía las posibilidades de análisis, pero también plantea cuestiones relacionadas con su calidad, estructura, procedencia y tratamiento. Kitchin (2014) analiza esta transformación del panorama de los datos y señala la necesidad de considerar de forma crítica tanto las posibilidades de Big Data y la analítica como las condiciones en las que los datos se obtienen y se utilizan.

La movilidad urbana se ha seleccionado como ámbito de aplicación porque proporciona un escenario adecuado para trabajar con información temporal procedente de múltiples puntos de medida. Los sistemas inteligentes de transporte han evolucionado hacia enfoques cada vez más orientados a los datos, en los que la información procedente de distintas fuentes puede procesarse para diferentes aplicaciones y agentes implicados (Zhang et al., 2011). En el caso concreto de este trabajo, el Ayuntamiento de Madrid proporciona datos abiertos históricos de tráfico y datos asociados a la ubicación de los puntos de medida, lo que permite disponer de una fuente pública y documentada para desarrollar el caso de estudio.

Desde el punto de vista académico, el proyecto permite aplicar conjuntamente conocimiento relacionados con la programación, el tratamiento de grandes volúmenes de registros, la

preparación de datos, el aprendizaje automático, la evaluación de modelos y el diseño de visualizaciones interactivas. El interés no reside únicamente en utilizar estas herramientas por separado, sino en integrarlas dentro de una solución software en la que las salidas de una fase constituyen las entradas necesarias para las siguientes.

Durante el desarrollo se han identificado además varios problemas técnicos que condicionan el resultado del análisis. Entre ellos se encuentran la interpretación de las variables de tráfico, la agregación de las mediciones a escala horaria, la existencia de periodos con información incompleta y la necesidad de utilizar en el modelo predictivo únicamente información disponible antes de la hora que se pretende estimar. Estas cuestiones han requerido definir criterios de preparación de los datos y un procedimiento de evaluación temporal coherente con el problema de predicción planteado.

Finalmente, el trabajo responde al interés por desarrollar una contribución práctica de tipo software cuya estructura, componentes y proceso de ejecución puedan documentarse de forma reproducible. La arquitectura se ha diseñado de forma modular, separando las responsabilidades de adquisición, procesamiento, modelado y visualización. Su evaluación se circunscribe al caso de estudio desarrollado con los datos de tráfico seleccionados y al entorno local utilizado, por lo que la aplicación efectiva de la arquitectura a otros dominios requeriría una validación específica.

1.2. Planteamiento del trabajo

El problema abordado en este TFM consiste en diseñar e implementar una solución que integre las fases necesarias para transformar datos históricos de tráfico de gran volumen en información preparada para su análisis predictivo y su visualización interactiva. La integración de estas etapas requiere definir las entradas y salidas de cada componente, las estructuras utilizadas para intercambiar información, los criterios aplicados durante el procesamiento y el orden de ejecución necesario para obtener los resultados finales.

Para dar respuesta a este problema se ha desarrollado una arquitectura modular organizada en componentes funcionales. El flujo comienza con los archivos originales, continúa con su ingesta y transformación, genera un conjunto de datos analítico, incorpora posteriormente el componente de modelo predictivo y finaliza con la preparación de las tablas utilizadas en

Power BI. La comunicación entre las distintas fases se realiza mediante archivos estructurados y una organización de directorios que permite diferenciar los datos originales, los resultados intermedios y las salidas destinadas a la visualización. La solución se ha implementado y ejecutado en un entorno local.

El caso práctico utiliza los datos históricos de tráfico de Madrid correspondientes a 2023 y 2024. El Ayuntamiento de Madrid publica estos datos históricos por meses y proporciona, además, conjuntos relacionados con la ubicación de los puntos de medida. A partir de estas fuentes, el procesamiento implementado selecciona los puntos incluidos en el caso de estudio, aplica los criterios de calidad definidos, construye las observaciones horarias e incorpora la información geográfica necesaria para el análisis. Posteriormente, el componente predictivo genera las variables temporales e históricas utilizadas por el modelo, establece una separación temporal entre entrenamiento y prueba y entrena un Random Forest Regressor. Su rendimiento se compara con dos modelos de referencia basados, respectivamente, en la intensidad de la hora anterior y en la intensidad correspondiente a la misma hora del día anterior mediante las métricas MAE, RMSE y R^2 .

El procesamiento incorpora comprobaciones destinadas a controlar la consistencia de los datos generados en las diferentes etapas. Entre ellas se encuentran la revisión de registros incompletos, la comprobación de la cobertura temporal y la aplicación del criterio de completitud definido para construir las observaciones horarias. La documentación del proyecto recoge asimismo la organización de los archivos y las dependencias necesarias para ejecutar los distintos componentes. La validación funcional de la solución y las pruebas realizadas sobre sus componentes se presentan posteriormente junto con el desarrollo técnico, de forma que puedan relacionarse con los requisitos definidos para el sistema.

El trabajo se enmarca en la modalidad Tipo 2 – Desarrollo de software, establecida para el TFM. En consecuencia, la contribución se centra en una solución tecnológica implementada para un problema concreto y en la evaluación de su funcionamiento dentro del caso de estudio. Este planteamiento requiere documentar los requisitos de la solución, justificar las principales decisiones de diseño y aportar evidencias de la ejecución y validación de los componentes desarrollados.

El alcance de la implementación queda delimitado por el caso de estudio, la selección de puntos de medida y los recursos del entorno local utilizado. No se ha implementado una

plataforma de procesamiento distribuido basada en Spark, un sistema de almacenamiento PostgreSQL ni un servicio de predicción en tiempo real. Estas alternativas quedan fuera de la solución evaluada. Del mismo modo, aunque la separación entre componentes facilita su posible adaptación, la reutilización efectiva de la arquitectura en otros dominios no se ha evaluado y requeriría una validación específica.

1.3. Estructura del trabajo

La memoria se organiza en ocho capítulos, además de las referencias bibliográficas y los anexos. La estructura permite presentar de forma diferenciada el contexto del problema, los objetivos y la metodología, el desarrollo de la solución tecnológica y los resultados obtenidos.

El capítulo 1 introduce el trabajo, presenta la motivación que justifica la elección del tema, delimita su planteamiento y describe la organización de la memoria.

El capítulo 2 desarrolla el contexto y el estado del arte relacionado con el trabajo. Se abordan las arquitecturas de datos masivos, el análisis predictivo aplicado a la movilidad urbana, la visualización interactiva y distintas herramientas y soluciones relacionadas. A partir de esta revisión se establece el contexto tecnológico en el que se sitúa la solución desarrollada.

El capítulo 3 presenta los objetivos y la metodología seguida durante el desarrollo. Se define el objetivo general y los objetivos específicos y se describen el enfoque metodológico, las fases del proyecto y los mecanismos utilizados para relacionar los objetivos planteados con las evidencias obtenidas.

El capítulo 4 aborda el marco normativo relacionado con el proyecto. Se consideran las condiciones aplicables a la utilización y reutilización de los datos públicos empleados y las cuestiones relacionadas con la protección de datos y la privacidad en función de la naturaleza de la información tratada.

El capítulo 5 recoge el desarrollo específico de la contribución y constituye el núcleo técnico de la memoria. Se describe la arquitectura implementada, las tecnologías y fuentes de datos utilizadas, los procesos de ingesta, limpieza, transformación y agregación, la construcción del conjunto de datos analítico, el modelado predictivo y su evaluación, así como la preparación y visualización de los resultados mediante Power BI.

El capítulo 6 está dedicado al código fuente y a los datos utilizados. Se presenta la organización del proyecto, los principales archivos y directorios, las dependencias y los elementos necesarios para comprender el proceso de ejecución y la generación de los resultados.

El capítulo 7 expone las conclusiones del trabajo y relaciona los resultados obtenidos con los objetivos planteados, sintetizando las principales aportaciones y el alcance de la solución desarrollada.

Por último, el capítulo 8 presenta las limitaciones y las posibles líneas de trabajo futuro. Se consideran las restricciones derivadas del caso de estudio y del entorno de ejecución, así como posibles ampliaciones de la arquitectura que no forman parte de la implementación evaluada.

2. Contexto y estado del arte

2.1.Contexto del problema

El aumento de la cantidad y diversidad de datos disponibles ha favorecido el desarrollo de nuevas formas de almacenamiento, procesamiento y análisis de la información. El concepto de Big Data no se refiere únicamente al volumen de los conjuntos de datos, sino también a características relacionadas con su generación, diversidad y tratamiento. Estas características condicionan las tecnologías y los procedimientos necesarios para transformar los datos originales en información susceptible de ser analizada.

El Big Data se ha caracterizado mediante distintas dimensiones destinadas a describir los principales problemas asociados al tratamiento de datos masivos. Entre las más utilizadas se encuentran el volumen, relacionado con la cantidad de datos; la velocidad, asociada a su ritmo de generación y procesamiento; la variedad, vinculada a la coexistencia de diferentes fuentes, estructuras y formatos; la veracidad, relacionada con la calidad y fiabilidad de la información; y el valor, entendido como la utilidad que puede obtenerse de su tratamiento. Esta caracterización permite considerar conjuntamente las propiedades de los datos y las necesidades que plantea su procesamiento, en lugar de identificar un problema de Big Data únicamente por el tamaño de los archivos.

El tratamiento de estos datos requiere organizar los componentes y los flujos que intervienen desde su obtención hasta su utilización analítica. En este trabajo, la arquitectura de datos se entiende como la organización de los procesos de adquisición, transformación, almacenamiento, modelado y preparación de resultados, junto con las relaciones establecidas entre ellos. Esta perspectiva permite separar responsabilidades y definir las entradas y salidas de cada fase de la solución implementada.

La movilidad urbana proporciona un ámbito de aplicación en el que se generan datos procedentes de diferentes infraestructuras y sistemas de transporte. Zhang et al. (2011) describen la evolución de los sistemas inteligentes de transporte hacia enfoques orientados a los datos, caracterizados por la disponibilidad de información procedente de múltiples fuentes y su procesamiento para diferentes aplicaciones y agentes. Los autores señalan asimismo cuestiones relacionadas con los componentes y el despliegue de este tipo de sistemas.

El caso de estudio desarrollado en este TFM utiliza datos publicados por el Portal de Datos Abiertos del Ayuntamiento de Madrid. El conjunto Tráfico. Histórico de datos del tráfico desde 2013 contiene mediciones históricas de los puntos de medida y presenta una frecuencia de actualización mensual. Su cobertura comienza el 1 de julio de 2013. El portal proporciona además un conjunto específico con la ubicación de los puntos de medida y otros recursos relacionados con el tráfico. Para este trabajo se han utilizado los archivos históricos correspondientes a 2023 y 2024 y la información geográfica necesaria para localizar los puntos seleccionados.

La disponibilidad de los archivos originales constituye únicamente el punto de partida del caso de estudio. Su utilización requiere interpretar la estructura y las variables proporcionadas por la fuente, aplicar los criterios de calidad definidos para el proyecto y transformar las mediciones originales hasta obtener las observaciones utilizadas en el análisis. En la implementación desarrollada, este proceso comprende la selección de los puntos de medida, la limpieza de los registros, la agregación a escala horaria, la generación de variables históricas y la preparación de los datos utilizados posteriormente por el modelo predictivo y el informe interactivo de Power BI.

La contribución de este TFM se sitúa en la integración de estas etapas dentro de una arquitectura modular implementada y evaluada mediante el caso de estudio. El objetivo no es proponer una arquitectura general aplicable a cualquier escenario de datos masivos, sino desarrollar una solución concreta y analizar su funcionamiento dentro del alcance definido. La posible adaptación de sus componentes a otros ámbitos se considera una posibilidad derivada de su diseño modular, pero no una capacidad demostrada experimentalmente en este trabajo.

2.2.Estado del arte

2.2.1. Arquitecturas Big Data para el análisis de datos masivos

Las arquitecturas orientadas al tratamiento de datos masivos pueden adoptar distintas configuraciones en función de las características de las fuentes, las necesidades de almacenamiento, la frecuencia de procesamiento y los requisitos de acceso a los resultados.

Por este motivo, la elección de una arquitectura implica decidir tanto la forma en que se almacenan los datos como el modelo de procesamiento utilizado.

En relación con el almacenamiento analítico, los data warehouses se han utilizado tradicionalmente para consolidar información procedente de sistemas operacionales y facilitar su consulta mediante herramientas de análisis e inteligencia empresarial. En este enfoque, los datos se organizan de acuerdo con un esquema orientado al consumo analítico. La evolución hacia fuentes más heterogéneas favoreció posteriormente la utilización de data lakes, basados en sistemas de almacenamiento capaces de conservar datos en formatos abiertos y hacerlos accesibles a diferentes motores y cargas de trabajo (Armbrust et al., 2021).

La separación entre data lakes y data warehouses aporta flexibilidad, pero también introduce la necesidad de mantener y sincronizar diferentes capas de almacenamiento y procesamiento. Armbrust et al. (2021) señalan problemas asociados a estas arquitecturas de dos niveles, como la duplicación de procesos ETL/ELT, la consistencia entre sistemas y el soporte de cargas de aprendizaje automático. Como respuesta a estas limitaciones, los autores presentan el concepto de lakehouse, que busca incorporar capacidades tradicionalmente asociadas a los sistemas de gestión de datos sobre almacenamiento basado en formatos abiertos y ofrecer soporte tanto para cargas analíticas como para ciencia de datos y aprendizaje automático.

De forma simplificada, las principales diferencias entre estos enfoques pueden resumirse de la siguiente manera:

Característica	Data warehouse	Data lake	Lakehouse
Orientación principal	Consulta y análisis de datos estructurados.	Almacenamiento flexible de datos para diferentes cargas de trabajo.	Integración de almacenamiento abierto y capacidades de gestión analítica.
Organización de los datos	Esquemas preparadis para el consumo analítico.	Datos almacenados en formatos abiertos y accesibles por distintos motores.	Formatos abiertos con funciones adicionales de gestión y rendimiento.

Cargas de trabajo	Principalmente inteligencia empresarial y análisis	Analítica, procesamiento y ciencia de datos	Inteligencia empresarial, analítica avanzada y aprendizaje automático
Aspecto relevante	Estructura optimizada para el análisis	Flexibilidad y acceso directo a los datos	Unificación de capacidades previamente distribuidas entre lake y warehouse.
Limitación o reto	Modelado y mantenimiento	Gobierno y organización de datos	Mayor complejidad de infraestructura

Tabla 1. Diferencias entre los distintos enfoques. Elaboración propia.

Otra decisión arquitectónica relevante es la forma en que se procesan los datos. El procesamiento por lotes opera sobre conjuntos de información disponibles para su ejecución y resulta apropiado cuando no existe un requisito de actualización inmediata. El procesamiento de flujos, por el contrario, trata información que continúa llegando al sistema y permite actualizar los resultados de manera incremental. En la actualidad existen motores que soportan ambos modelos. Apache Spark, por ejemplo, proporciona capacidades para procesamiento por lotes y streaming, y su API Structured Streaming permite expresar operaciones sobre flujos mediante abstracciones similares a las empleadas sobre datos estáticos.

La necesidad de combinar procesamiento histórico y resultados de baja latencia dio lugar a patrones arquitectónicos específicos. La arquitectura Lambda utiliza una capa batch para procesar el histórico y una capa de velocidad para incorporar los datos recientes, junto con una capa de servicio desde la que se consultan los resultados. Este planteamiento permite combinar ambos tipos de procesamiento, aunque implica mantener rutas diferentes para obtener las distintas vistas de los datos.

La arquitectura Kappa plantea una alternativa en la que el procesamiento se concentra en una única ruta basada en eventos. Cuando es necesario recalcular resultados históricos, los datos

conservados pueden reproducirse nuevamente a través del sistema de procesamiento. Este enfoque reduce la duplicación entre las rutas batch y streaming, aunque su aplicación depende de que el flujo de eventos pueda conservarse y reprocesarse de acuerdo con las necesidades del sistema.

La arquitectura implementada en este TFM responde a un escenario diferente. Los datos utilizados son archivos históricos mensuales correspondientes a un periodo cerrado y el objetivo experimental no requiere producir predicciones a medida que llegan nuevas mediciones. Por este motivo, se ha adoptado un procesamiento por lotes ejecutado en un entorno local. La solución separa las etapas de adquisición, ingesta y limpieza, preparación analítica, modelado predictivo y visualización, utilizando scripts de Python para el procesamiento, archivos estructurados para intercambiar resultados entre etapas y Power BI para su explotación visual.

La implementación desarrollada no corresponde, por tanto, a una arquitectura Lambda o Kappa ni utiliza un motor de procesamiento distribuido como Apache Spark. Tampoco se ha implementado un data lake, un lakehouse o un data warehouse empresarial. Estos enfoques permiten situar la solución desarrollada dentro del conjunto de alternativas existentes, pero incorporarlos al proyecto supondría modificar sus requisitos y su infraestructura. La evaluación realizada en este TFM se limita a la arquitectura local implementada y al caso de estudio definido.

2.2.2. Análisis predictivo aplicado a la movilidad urbana

La predicción del tráfico constituye una de las aplicaciones de los sistemas inteligentes de transporte y tiene como objetivo estimar el estado futuro de determinadas variables de movilidad a partir de la información disponible. Entre las variables estudiadas se encuentran el flujo, la velocidad y otros indicadores asociados al estado de la red. La naturaleza temporal y espacial del tráfico hace que su predicción dependa tanto de la evolución de las observaciones a lo largo del tiempo como de las características de los distintos puntos de la red (Jiang & Luo, 2022).

La literatura recoge diferentes familias de métodos para abordar este problema. Los enfoques tradicionales incluyen modelos estadísticos de series temporales, mientras que posteriormente se han incorporado métodos de aprendizaje automático y, de forma más

reciente, arquitecturas de aprendizaje profundo capaces de modelar relaciones temporales y espaciales más complejas. Jiang y Luo (2022) muestran esta evolución y revisan diferentes aproximaciones basadas en aprendizaje profundo aplicadas a la predicción de tráfico. La existencia de distintas familias de modelos refleja que la elección del método depende del tipo de datos disponible, de la variable que se desea predecir, del horizonte temporal y de las características del problema considerado.

Dentro de los métodos de aprendizaje supervisado se encuentra Random Forest, propuesto por Breiman (2001). Este método construye un conjunto de árboles a partir de mecanismos de aleatorización aplicados tanto a las observaciones utilizadas para entrenarlos como a las variables consideradas durante su construcción. En regresión, la predicción del conjunto se obtiene agregando las predicciones producidas por los árboles individuales. El procedimiento permite además obtener medidas de importancia de las variables, que pueden utilizarse para analizar su contribución dentro del modelo.

La utilización de información histórica exige prestar especial atención a la construcción de las variables predictoras. Los retardos o lags permiten representar valores observados en instantes anteriores, mientras que otras características pueden representar patrones asociados al calendario o resumir observaciones históricas mediante ventanas temporales. Estas transformaciones deben construirse exclusivamente a partir de información que estaría disponible en el instante en que se realiza la predicción. De lo contrario, puede producirse una fuga de información (data leakage), dando lugar a una evaluación que no representa correctamente el comportamiento esperado sobre observaciones futuras.

La estructura temporal de los datos también condiciona el procedimiento de evaluación. Una partición aleatoria puede mezclar observaciones anteriores y posteriores entre los conjuntos de entrenamiento y prueba. Para datos ordenados temporalmente, resulta necesario conservar el orden cronológico de las observaciones y evaluar el modelo sobre periodos posteriores a los utilizados para su entrenamiento. La documentación de scikit-learn establece este principio para la validación de series temporales y proporciona TimeSeriesSplit como estrategia específica para datos ordenados en el tiempo (Scikit-learn Developers, s. f.).

La evaluación de los modelos de regresión puede realizarse mediante métricas que describen distintos aspectos del error. El error absoluto medio (MAE) calcula la media de las diferencias absolutas entre los valores observados y las predicciones. La raíz del error cuadrático medio

(RMSE) se obtiene a partir de los errores elevados al cuadrado y, por esta razón, asigna un mayor peso a los errores de mayor magnitud. Por su parte, el coeficiente de determinación R^2 compara el ajuste del modelo con una referencia constante basada en el valor medio de la variable objetivo; un valor de 1 corresponde al mejor resultado posible, mientras que el indicador también puede adoptar valores negativos (Scikit-learn Developers, s. f.).

A partir de estas consideraciones, el modelo desarrollado en este TFM se plantea como un problema de regresión para estimar la intensidad horaria del tráfico. Se utiliza Random Forest Regressor con variables temporales e históricas construidas a partir de información anterior a la hora objetivo. La evaluación conserva el orden temporal de los datos y compara el modelo con dos referencias sencillas basadas en la intensidad de la hora anterior y en la correspondiente a la misma hora del día anterior. Los detalles de la construcción de las variables, la división de los conjuntos de entrenamiento, validación y prueba y los resultados obtenidos se presentan en el capítulo dedicado al desarrollo de la solución.

2.2.3. Visualización interactiva de datos de movilidad

La visualización interactiva permite explorar conjuntos de datos mediante representaciones gráficas cuyo contenido puede modificarse a partir de las acciones del usuario. Frente a una representación estática, este tipo de herramientas permite aplicar filtros, seleccionar elementos y consultar diferentes niveles de información, facilitando el análisis de un mismo conjunto de datos desde distintas perspectivas.

En el ámbito de la movilidad, la información presenta habitualmente componentes temporales y espaciales que condicionan su representación. Las series temporales permiten analizar la evolución de las variables a lo largo del tiempo, mientras que las representaciones geográficas permiten relacionar los indicadores con la localización en la que se han registrado. Estas representaciones pueden complementarse con gráficos comparativos, indicadores y tablas de detalle cuando resulta necesario examinar valores concretos.

La elección de cada representación debe estar relacionada con la naturaleza de las variables y con la finalidad del análisis. Además del tipo de gráfico utilizado, la interacción entre visualizaciones permite examinar subconjuntos de los datos sin modificar la fuente original. Las herramientas actuales de inteligencia empresarial incorporan mecanismos de filtrado, selección y navegación que permiten construir este tipo de experiencias analíticas.

Entre estas herramientas se encuentra Microsoft Power BI. Power BI Desktop permite conectar y preparar fuentes de datos, definir relaciones y modelos y construir informes compuestos por diferentes páginas y visualizaciones. Los informes admiten mecanismos de filtrado y segmentación que permiten modificar de forma interactiva la información mostrada (Microsoft, s. f.).

En este contexto conviene distinguir entre los conceptos de dashboard e informe utilizados por Microsoft. En Power BI, un dashboard es un lienzo de una única página disponible en Power BI Service y puede incorporar visualizaciones procedentes de distintos informes. Un informe, por el contrario, puede contener una o varias páginas y ofrece capacidades de filtrado, segmentación y exploración de los datos. Además, los dashboards no se crean en Power BI Desktop, mientras que los informes sí pueden desarrollarse en esta aplicación (Microsoft, s. f.).

La solución desarrollada en este TFM corresponde técnicamente a un informe interactivo multipágina de Power BI Desktop. El informe se ha organizado en ocho páginas destinadas a presentar la información general, la calidad de los datos, los análisis temporal y geográfico, los resultados del modelo predictivo, el análisis de sus errores y la arquitectura seguida. Las visualizaciones utilizan los resultados preparados previamente por los componentes de procesamiento y modelado, de modo que Power BI constituye la capa de modelado visual, consulta y presentación de la solución.

2.2.4. Herramientas y soluciones existentes

Las fases de una arquitectura de datos pueden implementarse mediante tecnologías con diferentes modelos de ejecución y niveles de abstracción. La elección depende, entre otros factores, del volumen de información, de los recursos disponibles y de si el procesamiento se realiza localmente o requiere distribuirse entre diferentes recursos computacionales. En el contexto de este trabajo resultan relevantes las tecnologías de procesamiento distribuido, las bibliotecas de análisis y aprendizaje automático y las plataformas de visualización interactiva.

Apache Spark es un motor de procesamiento que permite ejecutar cargas de trabajo de datos de forma distribuida. Su ecosistema incluye capacidades para el procesamiento de datos estructurados mediante Spark SQL y DataFrames, procesamiento de flujos mediante Structured Streaming y aprendizaje automático mediante MLlib (Apache Software

Foundation, s. f.). Estas características permiten utilizar una misma plataforma para diferentes cargas de procesamiento y la convierten en una alternativa para escenarios en los que se requiere ejecución distribuida.

Para el procesamiento local, Python dispone de bibliotecas especializadas en análisis y transformación de datos. pandas proporciona estructuras como DataFrame y funciones para importar, seleccionar, agrupar, transformar y exportar información estructurada. Determinadas operaciones de entrada permiten además trabajar de forma iterativa con archivos divididos en bloques, lo que puede utilizarse para controlar el volumen de datos cargado simultáneamente en memoria. Este mecanismo no constituye procesamiento distribuido, sino una estrategia de lectura y procesamiento dentro de un entorno local (The pandas development team, s. f.).

En el ámbito del aprendizaje automático, scikit-learn proporciona algoritmos de clasificación, regresión y agrupamiento, además de herramientas para preprocesamiento, selección de modelos y evaluación. Entre los estimadores disponibles se encuentra RandomForestRegressor, basado en la construcción y combinación de múltiples árboles de regresión. La biblioteca puede utilizar estructuras procedentes de NumPy y pandas, lo que facilita su incorporación a flujos de análisis desarrollados en Python (Scikit-learn Developers, s. f.).

La capa de visualización puede implementarse mediante plataformas de inteligencia empresarial como Power BI o Tableau. Ambas permiten construir representaciones interactivas y aplicar mecanismos de filtrado y exploración sobre los datos. Power BI Desktop incorpora además herramientas para la preparación de consultas, el modelado de datos y la creación de informes multipágina. Tableau, por su parte, permite construir vistas y dashboards interactivos en los que las selecciones realizadas sobre una visualización pueden utilizarse para filtrar otras vistas (Microsoft, s. f.; Tableau, s. f.).

La implementación de este TFM utiliza Python y pandas para las etapas de ingesta, limpieza, transformación y preparación de datos; scikit-learn para el modelado predictivo y su evaluación; archivos estructurados para intercambiar resultados entre las diferentes etapas; y Power BI Desktop para el modelado y la visualización interactiva. Apache Spark y Tableau se consideran tecnologías de referencia dentro de sus respectivos ámbitos, pero no forman parte de la solución implementada.

La elección de una solución local responde al alcance experimental del proyecto y al flujo de procesamiento definido. La contribución no se fundamenta en la creación de nuevas herramientas de procesamiento, aprendizaje automático o visualización, sino en su integración dentro de una arquitectura concreta aplicada al caso de estudio. Esta integración permite mantener diferenciadas las etapas del procesamiento, generar los datos utilizados por el modelo predictivo y trasladar posteriormente sus resultados a una capa de análisis visual.

Por tanto, la solución desarrollada no pretende sustituir plataformas distribuidas ni establecer una superioridad general de las tecnologías seleccionadas frente a otras alternativas. Su evaluación se limita al flujo implementado, a los datos utilizados y al entorno local en el que se ha ejecutado.

2.3.Conclusiones

La revisión realizada muestra que el diseño de una arquitectura de datos depende de las características de las fuentes, del modelo de procesamiento y de los requisitos de utilización de la información. Los enfoques basados en procesamiento por lotes y en flujos responden a necesidades diferentes, del mismo modo que los data warehouses, data lakes y lakehouses presentan distintas formas de organizar el almacenamiento y el acceso analítico. Por tanto, la incorporación de tecnologías distribuidas o de procesamiento continuo debe responder a los requisitos concretos del sistema y no únicamente al volumen de los datos.

En el ámbito de la movilidad urbana, la predicción de tráfico se ha abordado mediante diferentes familias de métodos, desde modelos estadísticos hasta técnicas de aprendizaje automático y aprendizaje profundo. La literatura revisada muestra asimismo la importancia de considerar las características temporales y espaciales del problema al seleccionar y evaluar los modelos. En particular, cuando se trabaja con información temporal, la construcción de las variables predictoras y la separación entre los datos de entrenamiento y evaluación deben evitar la utilización de información futura que no estaría disponible en el momento de realizar la predicción.

La visualización interactiva constituye una capa complementaria al procesamiento y al modelado, ya que permite consultar los resultados mediante representaciones y mecanismos

de interacción adaptados al tipo de información analizada. En los datos de movilidad, la combinación de información temporal y geográfica permite utilizar series temporales, mapas, indicadores y otros recursos visuales para examinar diferentes dimensiones del conjunto de datos y de los resultados del modelo.

El análisis de las tecnologías existentes muestra, además, que las diferentes etapas del ciclo de tratamiento de los datos pueden resolverse mediante herramientas especializadas o plataformas con capacidades integradas. En consecuencia, la contribución de este TFM no se plantea como la creación de una nueva tecnología de procesamiento, aprendizaje automático o visualización, sino como el diseño y la implementación de una arquitectura modular que integra herramientas existentes dentro de un flujo definido para el caso de estudio.

En concreto, la solución desarrollada procesa datos históricos de tráfico publicados por el Ayuntamiento de Madrid, construye un conjunto de observaciones horarias, incorpora un componente de predicción basado en Random Forest y prepara los resultados para su análisis mediante un informe interactivo en Power BI. La elección de un procesamiento por lotes y una ejecución local responde al carácter histórico de los datos utilizados y al alcance experimental establecido para el proyecto.

La evaluación realizada queda limitada al caso de estudio y al entorno de ejecución utilizados. Por este motivo, la escalabilidad mediante procesamiento distribuido, la automatización de la ingesta, el funcionamiento en tiempo real y la aplicación efectiva de la arquitectura a otros dominios no se consideran capacidades demostradas en este trabajo. Estas posibilidades constituyen líneas de ampliación que requerirían su propia implementación y evaluación.

3. Objetivos concretos y metodología de trabajo

Este capítulo presenta los objetivos del Trabajo Fin de Máster y describe el procedimiento seguido para desarrollar y evaluar la solución propuesta. A partir del contexto y del estado del arte expuestos en el capítulo anterior, la contribución se concreta en el diseño y la implementación de una arquitectura modular orientada al procesamiento de datos históricos, el análisis predictivo y la visualización interactiva.

El trabajo se enmarca en la modalidad Tipo 2 – Desarrollo de software. De acuerdo con la guía del TFM, esta modalidad aborda un problema concreto mediante el desarrollo o adaptación de una aplicación informática y sitúa la herramienta software programada como contribución central. La guía contempla asimismo una validación de la herramienta mediante pruebas de usabilidad y/o aceptación por parte de expertos. En consecuencia, el proyecto aborda tanto la implementación de los componentes de la solución como la evaluación de su funcionamiento dentro del caso de estudio.

3.1. Objetivo general

El objetivo general de este Trabajo Fin de Máster es diseñar, implementar y evaluar una arquitectura modular de datos que integre procesos de ingesta, preparación, análisis predictivo y visualización interactiva, mediante su aplicación a un caso de estudio basado en datos abiertos de tráfico de Madrid correspondientes al periodo 2023–2024.

La solución tiene como finalidad transformar los registros históricos originales en un conjunto de datos analítico, generar predicciones horarias a partir de información disponible previamente y presentar los datos y resultados obtenidos mediante un informe interactivo en Power BI. La implementación y evaluación se circunscriben al caso de estudio y al entorno local utilizados, por lo que no se plantea como objetivo demostrar escalabilidad distribuida, funcionamiento en tiempo real ni aplicabilidad efectiva a otros dominios.

3.2. Objetivos específicos

Para alcanzar el objetivo general se establecen los siguientes objetivos específicos:

OE1. Definir los requisitos y diseñar la arquitectura de la solución. Delimitar el alcance del sistema y establecer los componentes, responsabilidades, entradas y salidas necesarios para integrar las etapas de procesamiento, modelado predictivo y visualización.

OE2. Implementar el proceso de preparación de los datos. Obtener y documentar las fuentes abiertas utilizadas, aplicar los procesos de ingesta, selección, limpieza y agregación y generar un conjunto de observaciones horarias adecuado para las etapas posteriores del análisis.

OE3. Desarrollar y evaluar el componente de predicción horaria. Construir las variables predictoras a partir de información disponible previamente, entrenar un modelo Random Forest Regressor y evaluar su comportamiento mediante una separación temporal, métricas de regresión y modelos de referencia.

OE4. Diseñar e implementar un informe interactivo para el análisis de los datos y de los resultados. Integrar en Power BI la información generada por las etapas anteriores y proporcionar visualizaciones e interacciones que permitan explorar la calidad de los datos, su evolución temporal y geográfica y los resultados del modelo predictivo.

OE5. Verificar y documentar el funcionamiento de la solución. Comprobar las funcionalidades implementadas y la coherencia de las salidas generadas por los diferentes componentes, documentar el entorno y las dependencias necesarias para su ejecución y evaluar la solución mediante las pruebas definidas para el proyecto.

OE6. Analizar los resultados y delimitar el alcance de la solución. Relacionar las evidencias obtenidas con los objetivos planteados, identificar las principales limitaciones de los datos y de la implementación y establecer posibles líneas de ampliación que no hayan sido evaluadas en el desarrollo realizado.

Objetivo	Evidencia principal	Criterio de comprobación
OE1	Requisitos y arquitectura	Alcance, componentes, entradas y salidas documentados.

OE2	Datos procesados y controles de calidad	Procesamiento ejecutado y criterios de selección, limpieza, agregación y completitud documentados.
OE3	Modelo, predicciones y métricas	Evaluación temporal y comparación con modelos de referencia.
OE4	Archivo PBIX y evidencias del informe	Páginas, indicadores, filtros e interacciones implementados.
OE5	Código, documentación y resultados de las pruebas	Funcionalidades verificadas y procedimiento de ejecución documentado.
OE6	Resultados, conclusiones y limitaciones	Correspondencia entre objetivos, evidencias, limitaciones y líneas futuras.

Tabla 2. Objetivos específicos. Elaboración propia.

Los objetivos se han formulado de manera que su cumplimiento pueda evaluarse mediante resultados y evidencias identificables. La correspondencia entre cada objetivo, los elementos desarrollados y los resultados obtenidos se utilizará posteriormente para valorar su grado de cumplimiento en las conclusiones.

3.3. Metodología del trabajo

3.3.1. Enfoque metodológico

El desarrollo del TFM toma como referencia CRISP-DM (Cross-Industry Standard Process for Data Mining) para organizar las actividades relacionadas con la comprensión y preparación de los datos, el modelado y la evaluación. CRISP-DM estructura los proyectos de minería de datos en seis fases: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue. Estas fases no constituyen una secuencia estrictamente

lineal, ya que el modelo contempla la revisión de etapas anteriores cuando los resultados obtenidos requieren modificar decisiones previas (IBM, s. f.).

En este trabajo, CRISP-DM se utiliza como referencia para el componente analítico y se adapta al contexto de un TFM de desarrollo software. La comprensión del negocio se corresponde con la definición del problema y del alcance; las fases de comprensión y preparación de los datos se aplican a los históricos de tráfico; y las fases de modelado y evaluación comprenden la construcción y valoración del modelo predictivo. La fase de despliegue no se interpreta como un despliegue operativo en producción, sino como la preparación de los resultados y de la solución desarrollada para su ejecución, documentación y presentación en un entorno local.

El desarrollo se ha organizado además de forma incremental, abordando sucesivamente la definición del problema, la preparación de los datos, la implementación de la arquitectura, el modelado predictivo, la visualización y la evaluación de la solución. Durante este proceso fue necesario revisar etapas ya implementadas cuando las comprobaciones realizadas mostraron problemas en el tratamiento de los datos o en el planteamiento predictivo. En particular, se revisaron el procedimiento de agregación horaria y la disponibilidad temporal de las variables utilizadas por el modelo, lo que implicó modificar el procesamiento y repetir posteriormente el entrenamiento y la evaluación.

Este planteamiento permite combinar un marco específico para el trabajo con datos y modelos predictivos con las actividades propias del desarrollo de la solución software, sin considerar CRISP-DM como una metodología completa de ingeniería del software.

3.3.2. Fases del desarrollo

Fase 1. Definición del problema, alcance y requisitos

Se delimitó la contribución como una solución software modular y se seleccionó el tráfico de Madrid como caso de estudio. En esta fase se identificaron las funciones que debía proporcionar la solución, los recursos disponibles y las restricciones derivadas de una implementación ejecutada en un entorno local. Asimismo, se definieron los componentes principales y los criterios necesarios para comprobar posteriormente su funcionamiento.

Los requisitos se diferencian en funcionales y no funcionales. Los primeros describen las operaciones que debe realizar la solución, mientras que los segundos recogen condiciones de calidad y restricciones relacionadas con su utilización y mantenimiento. Como referencia para

la definición de atributos de calidad del producto software se considera el modelo ISO/IEC 25010:2023, que establece características aplicables a la especificación y evaluación de la calidad de productos software. Los requisitos concretos utilizados en el proyecto se presentan junto con el desarrollo técnico de la solución.

Fase 2. Comprensión y selección de los datos

Se analizaron los conjuntos de datos abiertos de tráfico publicados por el Ayuntamiento de Madrid y la información geográfica asociada a sus puntos de medida. Se seleccionó el periodo comprendido entre enero de 2023 y diciembre de 2024 y se revisaron las variables disponibles, su estructura y los formatos de los archivos antes de definir el procedimiento de procesamiento.

Para construir el prototipo se trabajó con una selección de 50 puntos de medida de acuerdo con el criterio aplicado en el pipeline. Esta selección define el alcance del caso de estudio y no constituye una muestra diseñada para garantizar la representatividad espacial de toda la red viaria de Madrid. Por este motivo, los resultados obtenidos se interpretan dentro del conjunto de puntos analizado.

Fase 3. Diseño e implementación de la arquitectura

Se diseñó una arquitectura local organizada en componentes de adquisición e ingesta, procesamiento de datos, preparación analítica, modelado predictivo y visualización. La implementación utiliza Python y pandas para el procesamiento, scikit-learn para el componente predictivo, archivos estructurados para el intercambio de resultados entre etapas y Power BI para la representación interactiva.

La solución se organizó mediante scripts diferenciados y una estructura de directorios destinada a separar los datos originales, los resultados intermedios y las salidas utilizadas posteriormente por otros componentes. Esta organización permite identificar las entradas y salidas de cada fase y mantener separadas sus responsabilidades. El procesamiento distribuido no forma parte de la implementación realizada.

Fase 4. Ingesta, limpieza y preparación del conjunto analítico

Se implementaron los procesos necesarios para leer los archivos mensuales, seleccionar los puntos incluidos en el caso de estudio, transformar las variables y construir observaciones a

escala horaria. El procedimiento de agregación se definió de acuerdo con la naturaleza de las variables contenidas en los archivos originales.

Para considerar completa una combinación sensor-hora se exigió la disponibilidad de las cuatro observaciones correspondientes a los minutos 00, 15, 30 y 45. Las combinaciones que no cumplieran este criterio se excluyeron del conjunto horario utilizado posteriormente. El procesamiento incorpora además comprobaciones relacionadas con la validez de las fechas, los registros duplicados y los valores utilizados en el análisis. Los resultados de estos controles y su efecto sobre el conjunto de datos se documentan en el capítulo de desarrollo.

Fase 5. Análisis exploratorio y modelado predictivo

A partir del conjunto horario se analizaron los patrones temporales y geográficos de los datos y se prepararon las variables utilizadas posteriormente por el modelo. Para la estimación de la intensidad horaria se construyeron características temporales e históricas por punto de medida, utilizando únicamente información anterior a la hora objetivo.

La evaluación se organizó cronológicamente. Los datos comprendidos entre enero y septiembre de 2023 se utilizaron para el entrenamiento inicial y los correspondientes a octubre-diciembre de 2023 para la validación. Tras esta fase, el modelo se reentrenó utilizando los datos de 2023 y se evaluó sobre el conjunto de prueba correspondiente a 2024. El Random Forest Regressor se comparó con dos modelos de referencia: uno basado en la intensidad de la hora anterior y otro en la intensidad registrada durante la misma hora del día anterior.

El rendimiento se evaluó mediante MAE, RMSE y R^2 y se complementó con el análisis de los errores. La separación cronológica permite que las observaciones utilizadas para evaluar el modelo sean posteriores a las empleadas durante su entrenamiento, evitando una partición aleatoria que pudiera introducir información futura en el proceso de evaluación.

Fase 6. Visualización interactiva

Los datos procesados y los resultados del modelo se prepararon en archivos estructurados para su incorporación a Power BI. A partir de estas salidas se desarrolló un informe interactivo multipágina que utiliza indicadores, gráficos temporales, representaciones geográficas, tablas, filtros y mecanismos de navegación.

Las distintas páginas permiten consultar aspectos relacionados con la calidad de los datos, el comportamiento temporal y geográfico del tráfico y los resultados del modelo predictivo. La definición concreta del modelo de datos, las medidas utilizadas y las interacciones disponibles se describe en el capítulo dedicado al desarrollo.

Fase 7. Evaluación, documentación y entrega

La evaluación de la solución se estructura en tres dimensiones. La dimensión técnica se orienta a comprobar el funcionamiento de los componentes implementados y la coherencia de sus entradas y salidas. La dimensión analítica evalúa el comportamiento del modelo predictivo mediante métricas de regresión y su comparación con los modelos de referencia. La dimensión de uso corresponde a la valoración del informe interactivo mediante el procedimiento de usabilidad o aceptación definido para el proyecto.

Para garantizar la trazabilidad de la evaluación, las pruebas realizadas se documentan indicando la funcionalidad o requisito evaluado, el procedimiento aplicado, el resultado obtenido y su valoración. La documentación del proyecto incluye asimismo los elementos necesarios para identificar el entorno de ejecución, las dependencias y la organización de los archivos utilizados por la solución.

La entrega incorpora el código fuente desarrollado, la documentación asociada al proceso de ejecución, la descripción de los datos utilizados y el archivo de Power BI correspondiente al informe interactivo. Estos elementos constituyen la base para la demostración práctica de la solución desarrollada.

3.3.3. Trazabilidad y control de la evaluación

La evaluación del proyecto se organiza mediante la relación entre los objetivos específicos, los elementos desarrollados y las evidencias obtenidas. Para cada objetivo se identifican los apartados de la memoria y los resultados que permiten valorar su grado de cumplimiento.

Las pruebas de la solución se documentan relacionando la funcionalidad o requisito evaluado con el procedimiento aplicado, el resultado obtenido y su valoración. Esta organización permite diferenciar entre las funcionalidades implementadas y aquellas cuyo funcionamiento ha sido comprobado mediante una evidencia concreta.

La trazabilidad entre objetivos, desarrollo y resultados se utiliza posteriormente en el capítulo de conclusiones para valorar el cumplimiento de los objetivos planteados. Las funcionalidades o capacidades que no hayan sido verificadas dentro del alcance experimental se presentan como limitaciones o posibles líneas de ampliación, sin considerarlas resultados demostrados del proyecto.

3.4. Uso de herramientas de inteligencia artificial

Durante la elaboración de este Trabajo Fin de Máster se ha utilizado ChatGPT, herramienta de inteligencia artificial generativa desarrollada por OpenAI, como recurso de apoyo en determinadas tareas del proyecto. Su utilización ha incluido la revisión de la estructura de la memoria, propuestas de reorganización y mejora de la redacción, apoyo en la revisión de scripts de Python y detección de posibles incoherencias entre diferentes partes del documento.

La herramienta también se ha empleado como apoyo en la preparación de algunos esquemas y representaciones gráficas relacionados con la arquitectura y la organización del proyecto. Estos usos se identifican específicamente en los elementos correspondientes cuando procede.

ChatGPT no se ha utilizado como fuente académica para fundamentar los contenidos teóricos, técnicos o normativos del trabajo. Las afirmaciones que requieren respaldo externo se fundamentan mediante bibliografía académica, documentación oficial o fuentes técnicas citadas de forma independiente. La ejecución del código, la obtención de los resultados y la comprobación de las métricas utilizadas en la memoria corresponden al desarrollo realizado para este TFM.

El Anexo G recoge con mayor detalle los principales usos de la herramienta durante la elaboración del proyecto. Asimismo, ChatGPT se incluye entre las referencias del trabajo de acuerdo con las indicaciones aplicables al uso de herramientas de inteligencia artificial.

4. Marco normativo

El desarrollo de este Trabajo Fin de Máster utiliza datos abiertos publicados por el Ayuntamiento de Madrid y genera nuevos conjuntos de información derivados de su procesamiento. Por este motivo, el marco normativo considerado se centra principalmente en la protección de datos personales y en las condiciones aplicables a la reutilización de información del sector público. Estas cuestiones permiten delimitar la naturaleza de los datos tratados y las condiciones bajo las que pueden utilizarse y transformarse dentro del proyecto.

4.1. Protección de datos personales

El marco general de protección de datos aplicable en España está constituido por el Reglamento (UE) 2016/679, Reglamento General de Protección de Datos (RGPD), y por la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. Esta última desarrolla y complementa determinados aspectos del marco europeo en el ordenamiento jurídico español.

El RGPD define como dato personal toda información relativa a una persona física identificada o identificable. Una persona puede considerarse identificable cuando su identidad pueda determinarse directa o indirectamente, incluso mediante datos de localización u otros identificadores. El Reglamento distingue además entre anonimización y seudonimización. Los datos seudonimizados continúan considerándose datos personales cuando pueden volver a atribuirse a una persona utilizando información adicional, mientras que los principios de protección de datos no se aplican a información verdaderamente anónima respecto de la cual el interesado ya no resulte identificable.

Los conjuntos utilizados en este TFM contienen mediciones agregadas de tráfico asociadas a puntos de medida de la red viaria de Madrid. Las variables empleadas describen características del tráfico y del sensor, como intensidad, ocupación, carga, velocidad y localización del punto de medida. En los datos tratados para la implementación no se utilizan nombres, documentos de identidad, matrículas, identificadores personales ni trayectorias individualizadas de personas o vehículos.

A partir de la información utilizada en el proyecto, no se realiza un tratamiento de datos personales identificables en el sentido del artículo 4 del RGPD. Esta conclusión se refiere únicamente a los conjuntos de datos efectivamente empleados en el TFM y no implica que cualquier información agregada o relacionada con movilidad quede automáticamente fuera del ámbito de la normativa. La consideración de un conjunto como anónimo depende de que no permita identificar razonablemente a una persona teniendo en cuenta los medios que puedan utilizarse para ello.

Aunque los datos tratados no contienen identificadores personales, el proyecto aplica criterios de tratamiento limitado a la información necesaria para el objetivo definido y mantiene documentadas las transformaciones realizadas sobre las fuentes originales. Estas prácticas son coherentes con principios como la minimización y la limitación de la finalidad recogidos en el RGPD.

4.2. Reutilización de información del sector público

La reutilización de información producida o custodiada por organismos del sector público se encuentra regulada en España por la Ley 37/2007, de 16 de noviembre, sobre reutilización de la información del sector público. La norma establece el régimen jurídico aplicable a la reutilización de documentos que se encuentran en poder de las administraciones y organismos incluidos en su ámbito de aplicación.

A estos efectos, la reutilización comprende el uso de documentos del sector público por personas físicas o jurídicas para fines distintos de la actividad administrativa para la que fueron producidos. Las condiciones concretas de reutilización dependen de la modalidad mediante la que cada organismo ponga la información a disposición y de las condiciones asociadas al conjunto utilizado.

El Portal de Datos Abiertos del Ayuntamiento de Madrid permite, con carácter general, la reutilización de los documentos sometidos a sus condiciones para fines comerciales y no comerciales. Entre las condiciones aplicables se encuentran la obligación de citar la fuente, no desnaturalizar el sentido de la información, conservar determinados metadatos asociados a la actualización y a las condiciones de reutilización y evitar cualquier indicación que pueda sugerir que el Ayuntamiento patrocina o respalda la reutilización realizada.

El portal identifica además la licencia Creative Commons Attribution 4.0 International (CC BY 4.0) como una de las modalidades utilizadas para la publicación de sus conjuntos de datos.

En este TFM se han reutilizado conjuntos de datos de tráfico publicados por el Ayuntamiento de Madrid con finalidad académica. En la memoria se identifica el organismo de procedencia y los conjuntos utilizados, y los resultados generados por los procesos de limpieza, agregación, transformación y modelado se presentan como elaboraciones derivadas de las fuentes originales. Esta distinción permite diferenciar los datos publicados oficialmente por el Ayuntamiento de los conjuntos y resultados obtenidos durante el desarrollo del proyecto.

Las condiciones generales del portal establecen asimismo que, cuando la información haya sido anonimizada por razones de protección de datos, no pueden realizarse labores de reidentificación mediante su combinación con otras fuentes. Aunque esta circunstancia no constituye una operación realizada en el presente TFM, forma parte de las condiciones generales de reutilización aplicables a los datos publicados por el Ayuntamiento.

4.3. Normativa relacionada con movilidad

El caso de estudio se sitúa en el ámbito de la movilidad urbana, por lo que resulta pertinente considerar la Ley 9/2025, de 3 de diciembre, de Movilidad Sostenible. Esta norma establece el marco general de la movilidad sostenible en España e incorpora, entre otros aspectos, disposiciones relacionadas con la digitalización y la disponibilidad de información sobre transporte y movilidad.

La ley concede relevancia al acceso y disponibilidad de datos como elementos asociados a la digitalización del sistema de transporte. En particular, regula la puesta a disposición de determinados datos relativos a servicios, infraestructuras y equipamientos de transporte en condiciones de acceso estandarizado e interoperable y contempla la publicación de información relacionada con la movilidad.

Este marco guarda relación con el caso de estudio del TFM en la medida en que el proyecto utiliza información pública de tráfico para realizar un análisis histórico y desarrollar una solución de procesamiento, predicción y visualización. No obstante, la arquitectura desarrollada no constituye un sistema operativo de gestión del tráfico ni interviene sobre infraestructuras de transporte.

La solución no controla semáforos, no modifica rutas ni actúa sobre dispositivos de la red viaria. Tampoco se ha desplegado como servicio de predicción en tiempo real ni adopta decisiones automatizadas con efectos sobre personas. Por tanto, la Ley 9/2025 se utiliza principalmente como marco normativo y contextual del dominio de movilidad en el que se sitúa el caso de estudio, y no como regulación específica de un sistema de control implantado.

4.4. Consideraciones éticas y de seguridad

La disponibilidad pública de un conjunto de datos no elimina la necesidad de valorar los posibles riesgos derivados de su utilización y de su combinación con otras fuentes. En particular, cuando se trabaja con información susceptible de relacionarse con localizaciones o movimientos, debe evitarse que nuevas combinaciones de datos permitan obtener información sobre personas que no estuviera presente de forma directa en las fuentes originales.

En el caso desarrollado, los datos utilizados corresponden a mediciones agregadas de tráfico y a información asociada a los puntos de medida. El proyecto no incorpora identificadores personales, matrículas ni información destinada a reconstruir trayectorias individuales. Asimismo, no se han realizado procesos destinados a identificar personas a partir de los datos utilizados.

Desde el punto de vista de la gestión de la información, los archivos originales se conservan separados de los datos obtenidos tras las etapas de limpieza, agregación y transformación. Esta separación permite distinguir la información publicada por la fuente oficial de los resultados generados durante el procesamiento y facilita la trazabilidad de las transformaciones realizadas.

La documentación del proyecto identifica las fuentes utilizadas y describe las principales operaciones aplicadas sobre los datos. Los conjuntos y resultados derivados se presentan como productos del procesamiento realizado en el TFM y no como información oficial elaborada por el Ayuntamiento de Madrid.

Estas medidas responden al alcance académico y experimental de la solución y buscan mantener una utilización responsable y trazable de los datos empleados.

4.5. Aplicación al TFM

El marco normativo analizado afecta al proyecto principalmente en tres aspectos. En primer lugar, los conjuntos utilizados contienen información agregada de tráfico asociada a puntos de medida y, en la forma en que se han utilizado en este trabajo, no incorporan identificadores personales ni trayectorias individualizadas. Esta valoración se limita a las fuentes y variables efectivamente tratadas y no puede generalizarse a cualquier conjunto de datos relacionado con movilidad.

En segundo lugar, los datos proceden del Portal de Datos Abiertos del Ayuntamiento de Madrid y su utilización debe respetar las condiciones de reutilización asociadas a la fuente. Por este motivo, la memoria identifica la procedencia de los conjuntos empleados y diferencia los datos originales de los resultados obtenidos mediante los procesos de limpieza, transformación, agregación y modelado.

En tercer lugar, la arquitectura desarrollada se limita a una herramienta académica de procesamiento, predicción y visualización sobre datos históricos. No se ha implantado como sistema de control del tráfico ni como servicio operativo en tiempo real y no realiza actuaciones automáticas sobre la movilidad o sobre personas.

En consecuencia, el marco normativo del proyecto se concreta en la utilización responsable de los datos, el respeto de sus condiciones de reutilización y la delimitación del alcance de la solución desarrollada respecto de sistemas operativos de gestión de movilidad.

5. Desarrollo específico de la contribución

Este capítulo describe el diseño, la implementación y la evaluación de la solución desarrollada para integrar el procesamiento de datos, el análisis predictivo y la visualización interactiva dentro de un mismo flujo de trabajo. La contribución se concreta en una arquitectura modular implementada en un entorno local y aplicada a un caso de estudio basado en datos históricos de tráfico urbano de Madrid.

La solución comprende la adquisición y organización de los datos originales, los procesos de ingesta, limpieza y transformación implementados en Python, la construcción del conjunto de datos analítico, el modelado predictivo y la preparación de los resultados para su análisis mediante Power BI. Los diferentes componentes intercambian sus resultados mediante archivos estructurados y se organizan de forma que puedan identificarse las entradas, operaciones y salidas correspondientes a cada etapa.

La evaluación realizada se circunscribe al caso de estudio, al periodo analizado y al entorno local utilizado. Por este motivo, el procesamiento distribuido, el funcionamiento en tiempo real y la reutilización efectiva de la arquitectura en otros dominios no se consideran capacidades demostradas en este trabajo.

5.1. Descripción general de la arquitectura propuesta

La arquitectura implementada se organiza en seis componentes funcionales: adquisición de datos, ingesta, procesamiento y limpieza, preparación del conjunto de datos analítico, modelado predictivo y visualización interactiva. Cada componente recibe unas entradas determinadas, realiza las operaciones correspondientes a su función y genera resultados que pueden ser utilizados por las etapas posteriores.

El flujo se ejecuta de forma secuencial y mediante procesamiento por lotes sobre archivos históricos. Los datos originales se mantienen separados de los conjuntos generados durante las sucesivas transformaciones, lo que permite identificar la procedencia de las salidas utilizadas por el modelo predictivo y por Power BI. La implementación no utiliza procesamiento distribuido ni incorpora un flujo de datos en tiempo real.

Diseño e implementación de una arquitectura de datos masivos para el análisis predictivo y la visualización interactiva

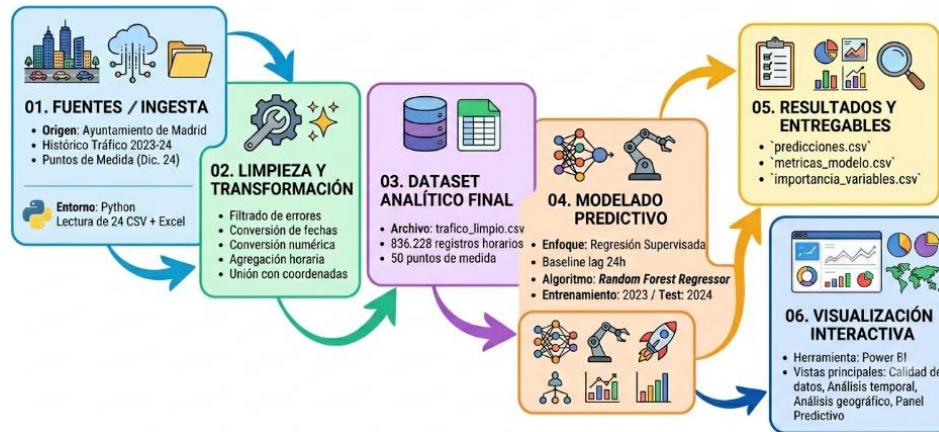


Figura 1. Arquitectura general propuesta para el análisis predictivo y la visualización interactiva. Elaboración propia

Componente	Entrada principal	Función	Salida principal	Tecnología
Adquisición	Fuentes abiertas del Ayuntamiento	Obtención de datos históricos y metadatos geográficos	Archivos originales	Portal de Datos Abiertos de Madrid
Ingesta	Archivos originales	Lectura, selección y tipado inicial	Registros seleccionados	Python, pandas
Procesamiento y limpieza	Registros seleccionados	Filtrado, controles de calidad y agregación horaria	Datos horarios procesados	Python, pandas
Preparación analítica	Datos horarios	Construcción de variables temporales e históricas	Dataset analítico	Python, pandas
Modelado predictivo	Dataset analítico	Entrenamiento, predicción y evaluación	Predicciones y métricas	Scikit-learn

Visualización	Datos procesados, predicciones y métricas	Modelado visual y exploración interactiva	Informe Power BI	Power BI Desktop
---------------	---	---	------------------	------------------

Tabla 3. Componentes principales de la arquitectura implementada. Elaboración propia.

La arquitectura se basa en componentes accesibles y de uso general. Esto facilita la reproducción del entorno académico y permite inspeccionar las salidas de cada etapa. Sin embargo, esta elección implica que el sistema está limitado por los recursos del equipo local y no debe equipararse a una arquitectura distribuida empresarial.

5.2. Requisitos de la solución

Los requisitos de la solución se han definido a partir de las funciones implementadas en los diferentes componentes de la arquitectura y de las condiciones necesarias para ejecutar el flujo de procesamiento en el entorno local utilizado. Se distinguen requisitos funcionales, relacionados con las operaciones que debe realizar el sistema, y requisitos no funcionales, asociados a características y restricciones de la implementación.

La definición de estos requisitos permite relacionar las funcionalidades descritas en los apartados siguientes con evidencias concretas de implementación y establecer posteriormente las pruebas utilizadas para comprobar su funcionamiento.

5.2.1. Requisitos funcionales

Los requisitos funcionales describen las operaciones que debe realizar la solución desde la lectura de las fuentes originales hasta la generación de los archivos utilizados para la visualización.

Requisito	Requisito funcional	Criterio de aceptación
RF01	El sistema debe permitir inspeccionar la estructura de los	Se identifican los archivos disponibles y se muestran sus columnas y una muestra de registros.

	archivos originales antes de su procesamiento.	
RF02	El sistema debe leer los archivos históricos de tráfico y la información geográfica de los puntos de medida.	Los archivos requeridos se localizan y pueden ser procesados por los componentes correspondientes.
RF03	El sistema debe seleccionar los 50 puntos de medida utilizados en el caso de estudio conforme al criterio implementado.	Se genera el conjunto de 50 identificadores seleccionados y se almacena su información.
RF04	El sistema debe aplicar los controles de limpieza y calidad definidos sobre los registros de tráfico.	Se descartan registros con error, fechas o intensidades no válidas e intensidades negativas, y se detectan situaciones incompatibles con las reglas definidas.
RF05	El sistema debe generar observaciones horarias completas a partir de las mediciones originales.	Solo se conservan combinaciones punto-hora con cuatro registros y con la frecuencia temporal esperada.
RF06	El sistema debe incorporar a las observaciones procesadas la información temporal y geográfica necesaria para el análisis.	El conjunto procesado contiene las variables temporales generadas y la información disponible de los puntos de medida.
RF07	El sistema debe construir las variables predictoras utilizando información temporal e histórica anterior a la hora objetivo.	Se generan retardos, medias móviles y variables operativas retardadas por punto de medida antes del entrenamiento.
RF08	El sistema debe separar cronológicamente los datos	Los datos anteriores a octubre de 2023 se destinan al entrenamiento inicial, octubre-diciembre de 2023 a validación

	utilizados para entrenamiento, validación y prueba.	y los datos desde enero de 2024 a prueba.
RF09	El sistema debe entrenar y evaluar un modelo Random Forest Regressor y compararlo con modelos de referencia.	Se generan MAE, RMSE y R^2 para Random Forest, persistencia de una hora y baseline de 24 horas.
RF10	El sistema debe generar y almacenar las predicciones y los resultados necesarios para analizar el modelo.	Se exportan predicciones, errores, métricas, importancia de variables y resumen de las particiones utilizadas.
RF11	El sistema debe preparar los conjuntos de datos utilizados por la capa de visualización.	Se generan las tablas CSV específicas requeridas para su incorporación a Power BI.
RF12	El informe interactivo debe permitir consultar los datos procesados y los resultados del caso de estudio.	El archivo de Power BI carga las tablas preparadas y permite consultar los indicadores y visualizaciones definidos para el proyecto.

Tabla 4. Requisitos funcionales de la solución. Elaboración propia

5.2.2. Requisitos no funcionales

Requisito	Requisito no funcional	Criterio de aceptación
RNF01	La solución debe poder ejecutarse en el entorno local definido para el proyecto utilizando las dependencias software necesarias.	El entorno dispone de las bibliotecas utilizadas por los componentes y permite ejecutar el flujo implementado.
RNF02	La implementación debe mantener separadas las responsabilidades principales del procesamiento.	La ingesta y limpieza, el modelado predictivo y la preparación de datos para Power BI se encuentran

		implementados en componentes diferenciados.
RNF03	Los datos originales deben mantenerse separados de las salidas procesadas.	La estructura de directorios utiliza ubicaciones diferenciadas para datos originales y resultados generados.
RNF04	El procesamiento de los archivos históricos debe limitar la cantidad de registros leídos simultáneamente durante la ingesta.	Los archivos históricos se leen mediante bloques de tamaño configurado durante las operaciones de selección y procesamiento.
RNF05	El flujo debe detectar determinadas condiciones de datos incompatibles con el procesamiento definido.	La ejecución genera errores explícitos ante ausencia de archivos necesarios, duplicados incompatibles, frecuencias temporales inesperadas o particiones temporales vacías.
RNF06	Las salidas de cada componente deben almacenarse en formatos estructurados que puedan ser utilizados por las etapas posteriores.	Los componentes generan archivos CSV con estructura definida que son consumidos por los siguientes procesos.

Tabla 5. Requisitos no funcionales de la solución. Elaboración propia

5.3. Instanciación de la arquitectura en el caso de la movilidad urbana

Para aplicar y evaluar la arquitectura desarrollada se ha utilizado como caso de estudio el tráfico urbano de Madrid. El Portal de Datos Abiertos del Ayuntamiento de Madrid publica un conjunto histórico de datos procedentes de los puntos de medida de tráfico, actualizado mensualmente y disponible desde julio de 2013. El portal proporciona además documentación sobre la estructura de los ficheros y un conjunto de datos relacionado con la ubicación de los puntos de medida.

El caso de estudio se ha delimitado al periodo comprendido entre enero de 2023 y diciembre de 2024, para el que se han utilizado 24 archivos mensuales. Los registros de tráfico se han complementado con la información de los puntos de medida para incorporar al conjunto procesado las variables geográficas disponibles, entre ellas el distrito, la longitud y la latitud.

La implementación se ha limitado a 50 puntos de medida. La selección se realiza a partir del primer archivo histórico procesado, ordenando los puntos según el número de registros identificados como válidos y conservando los 50 con mayor número de observaciones. Este procedimiento permite reducir el volumen procesado en el entorno local, pero no constituye un muestreo diseñado para garantizar la representatividad espacial de la red viaria de Madrid. En consecuencia, los puntos seleccionados no se consideran representativos de todos los distritos, tipos de vía o condiciones de circulación.

El problema predictivo consiste en estimar la intensidad de tráfico correspondiente a una hora determinada y a un punto de medida concreto utilizando variables temporales e información histórica disponible en instantes anteriores. La unidad de análisis utilizada para el modelado es, por tanto, la combinación de punto de medida y hora.

Para construir esta unidad analítica, las observaciones originales se agregan a nivel horario. La intensidad horaria se calcula mediante la media de las mediciones disponibles dentro de cada hora y únicamente se conservan las horas que cumplen el criterio de completitud definido en el proceso de ingesta. La misma agregación calcula valores medios de ocupación, carga, velocidad y periodo de integración.

La granularidad horaria reduce el volumen de observaciones respecto a los datos originales y proporciona una unidad temporal común para la preparación de variables históricas, el modelado predictivo y el análisis posterior de los resultados.



Figura 2. Arquitectura aplicada al caso práctico de predicción de intensidad del tráfico urbano de Madrid. Elaboración propia.

5.4. Tecnologías seleccionadas y justificación

La implementación se ha realizado principalmente mediante Python, pandas, scikit-learn y Power BI Desktop. Los diferentes componentes intercambian sus resultados mediante archivos CSV, de forma que las salidas generadas durante el procesamiento puedan ser utilizadas posteriormente por el modelado predictivo y la capa de visualización.

Python se utiliza como lenguaje principal para implementar los scripts de inspección, ingesta, limpieza, preparación de datos, modelado predictivo y exportación de resultados. La separación de estas funciones en scripts diferenciados permite ejecutar y revisar de forma independiente las principales etapas del procesamiento.

pandas se emplea para la lectura y transformación de los datos tabulares, incluyendo la conversión de tipos, el filtrado de registros, las agrupaciones, la generación de variables y la exportación de resultados. Debido al volumen de los archivos históricos, determinadas operaciones de ingesta se realizan mediante lectura por bloques. En la implementación se ha establecido un tamaño de 500.000 registros por bloque para evitar la carga simultánea del archivo histórico completo en memoria. Este valor constituye una decisión de implementación utilizada en el entorno local del proyecto y no se plantea como un valor óptimo general.

scikit-learn se utiliza en el componente predictivo para implementar RandomForestRegressor y calcular las métricas utilizadas durante la evaluación. El modelo se entrena a partir de las variables preparadas previamente y sus resultados se comparan con dos estrategias de referencia temporales mediante MAE, RMSE y R^2 .

Power BI Desktop constituye la capa de visualización de la arquitectura. Los resultados generados por los scripts se incorporan a un informe interactivo multipágina que integra visualizaciones temporales y geográficas, indicadores de calidad y resultados del modelo predictivo. Power BI Desktop permite construir informes con varias páginas y trabajar con visualizaciones, consultas, relaciones y modelos de datos. En este proyecto no se utiliza para entrenar el modelo predictivo, sino para consultar y representar los resultados previamente generados.

Los **archivos CSV** se utilizan como mecanismo de intercambio entre los componentes de la arquitectura. Esta decisión permite inspeccionar directamente las salidas intermedias y proporciona compatibilidad con las herramientas utilizadas. No obstante, la solución no incorpora una base de datos ni un formato columnar específicamente diseñado para escenarios distribuidos, por lo que esta elección se circunscribe al prototipo local desarrollado.

Apache Spark y PostgreSQL se analizaron como tecnologías alternativas durante el diseño, pero no forman parte de la implementación ejecutada. Por tanto, el procesamiento distribuido y el almacenamiento mediante un sistema gestor de bases de datos quedan fuera del alcance experimental del proyecto.

Tecnología	Uso en la implementación	Justificación en el proyecto
Python	Ejecución de los componentes del pipeline.	Permite integrar las diferentes etapas implementadas mediante scripts.
pandas	Lectura, limpieza, agregación y preparación de datos.	Proporciona las operaciones tabulares utilizadas y permite lectura por bloques.
scikit-learn	Entrenamiento y evaluación del modelo.	Proporciona Random Forest Regressor y las métricas de regresión utilizadas.
Power BI Desktop	Informe interactivo multipágina.	Permite integrar visualizaciones, indicadores, filtros y resultados analíticos.
CSV	Intercambio de datos entre componentes.	Formato compatible con Python y Power BI y directamente inspeccionable.

Tabla 6. Tecnologías utilizadas en la implementación. Elaboración propia.

5.5.Fuentes de datos utilizadas

El caso de estudio utiliza dos conjuntos de datos publicados por el Portal de Datos Abiertos del Ayuntamiento de Madrid. Ambos proporcionan la información necesaria para combinar las mediciones históricas de tráfico con la localización de los puntos de medida.

La fuente principal es Tráfico. Histórico de datos del tráfico desde 2013, que contiene las mediciones registradas en los puntos de medida de tráfico de Madrid. El conjunto se actualiza mensualmente mediante la incorporación de los datos correspondientes al mes anterior y dispone de información histórica desde julio de 2013. El portal proporciona además documentación específica sobre el contenido y la estructura de los ficheros.

Para este TFM se han utilizado los archivos mensuales comprendidos entre enero de 2023 y diciembre de 2024, lo que supone un total de 24 ficheros históricos en formato CSV.

La segunda fuente es Tráfico. Ubicación de los puntos de medida del tráfico. Este conjunto está relacionado con el histórico y proporciona información descriptiva y geográfica de los puntos de medida. El Portal de Datos Abiertos permite obtener estos datos en distintos formatos, entre ellos CSV y XLSX. En la implementación se ha utilizado el archivo XLSX correspondiente a diciembre de 2024.

Fuente	Formato utilizado	Periodo utilizado	Función en el proyecto
Tráfico. Histórico de datos del tráfico desde 2013.	CSV mensual	Enero 2023 - diciembre 2024	Mediciones de tráfico y marcas temporales.
Tráfico. Ubicación de los puntos de medida del tráfico.	XLSX	Diciembre 2024	Información descriptiva y geográfica de los puntos.

Tabla 7. Fuentes de datos utilizadas en el caso de estudio. Elaboración propia a partir de datos del Ayuntamiento.

Los archivos históricos procesados contienen los campos id, fecha, tipo_elem, intensidad, ocupacion, carga, vmed, error y periodo_integracion. Estas son las variables seleccionadas expresamente durante la lectura de los archivos históricos por el proceso de ingesta. La

interpretación de los campos se realiza de acuerdo con la documentación de contenido y estructura proporcionada por el Ayuntamiento de Madrid.

El conjunto de ubicación contiene información asociada a los puntos de medida. En la implementación se utilizan los campos disponibles entre id, tipo_elem, distrito, nombre, longitud y latitud. El identificador id actúa como campo común para relacionar la información geográfica con las observaciones de tráfico. Las coordenadas de longitud y latitud permiten posteriormente representar los puntos en la capa geográfica del informe, mientras que el nombre y el distrito proporcionan información descriptiva adicional.

5.6. Proceso de ingesta y caracterización de los datos

El proceso de ingesta se ha implementado mediante el script 01_ingesta_limpieza.py. Este componente localiza los archivos mensuales almacenados en el directorio de datos originales, procesa las mediciones de tráfico, selecciona los puntos incluidos en el caso de estudio y genera los conjuntos utilizados en las etapas posteriores.

Los archivos históricos se leen secuencialmente mediante bloques de 500.000 registros. La lectura por bloques evita cargar simultáneamente en memoria la totalidad de cada archivo y permite adaptar el procesamiento al entorno local utilizado. Este mecanismo no constituye procesamiento distribuido, ya que las operaciones continúan ejecutándose en un único entorno local.

La selección de los puntos de medida se realiza utilizando el primer archivo histórico procesado como referencia. El script contabiliza, para cada identificador, los registros cuyo indicador error presenta el valor N, ordena los puntos de mayor a menor número de registros válidos y conserva los 50 primeros. El resultado de esta selección se almacena para mantener identificados los puntos incluidos en el caso de estudio.

Este criterio tiene como finalidad obtener un conjunto de puntos con disponibilidad suficiente de observaciones para ejecutar el pipeline y evaluar la solución en el entorno disponible. No constituye un muestreo espacial estratificado y, por tanto, los 50 puntos seleccionados no se consideran representativos de toda la red viaria de Madrid.

Durante la ejecución se han procesado los 24 archivos mensuales comprendidos entre enero de 2023 y diciembre de 2024. El conjunto de archivos contiene 308.458.911 registros originales. Tras restringir el procesamiento a los puntos seleccionados y aplicar los controles iniciales de validez definidos en el pipeline, se conservaron 3.339.435 registros para las operaciones de agregación.

Posteriormente, las mediciones se agruparon por punto de medida y hora. El proceso generó finalmente 833.086 observaciones horarias completas y excluyó 3.142 combinaciones punto-hora que no cumplían el criterio de completitud establecido. La reducción entre los registros seleccionados y las observaciones finales responde principalmente a la transformación de las mediciones originales a granularidad horaria y a la exclusión de las horas incompletas.

Indicador	Resultado
Periodo analizado	Enero 2023-diciembre 2024
Archivos históricos procesados	24
Registros originales leídos	308.458.911
Puntos de medida seleccionados	50
Registros conservados tras los filtros iniciales	3.339.435
Horas incompletas excluidas	3.142
Observaciones horarias finales	833.086

Tabla 8. Resumen cuantitativo del proceso de ingesta y preparación. Elaboración propia.

5.7.Limpieza, transformación y agregación horaria

La fase de limpieza tiene como finalidad transformar los registros seleccionados en un conjunto coherente para las etapas posteriores de análisis. El proceso filtra los registros correspondientes a los puntos de medida seleccionados y conserva aquellos cuyo campo error presenta el valor N. A continuación, el campo fecha se convierte a formato temporal y las variables intensidad, ocupacion, carga, vmed y periodo_integracion se convierten a formato

numérico. Se eliminan los registros sin identificador, fecha o intensidad válida, así como aquellos que presentan valores negativos de intensidad.

Antes de realizar la agregación, el proceso comprueba la existencia de registros duplicados para una misma combinación de punto de medida y marca temporal. Si se detectan duplicados, la ejecución se detiene para evitar que estos registros intervengan en el cálculo de los valores horarios.

Posteriormente, las observaciones se agrupan por punto de medida y hora. Para cada combinación se contabiliza el número de registros disponibles y se conservan únicamente aquellas que contienen cuatro observaciones. Además, el proceso comprueba que los minutos de las marcas temporales procesadas pertenezcan a los valores esperados 00, 15, 30 o 45. Las combinaciones punto-hora que no contienen cuatro registros se consideran incompletas y se excluyen del conjunto horario.

Mediante este procedimiento se excluyeron 3.142 combinaciones punto-hora incompletas y se obtuvo un conjunto final de 833.086 observaciones horarias. El criterio aplicado evita calcular los indicadores horarios a partir de grupos con distinto número de registros y mantiene una regla homogénea de agregación para las observaciones conservadas.

La intensidad horaria se calcula mediante la media de las observaciones de intensidad disponibles en cada combinación punto-hora. El mismo criterio de agregación mediante la media se aplica a la ocupación, la carga, la velocidad y el periodo de integración. Adicionalmente, se conserva el número de registros utilizado en cada agrupación mediante la variable registros_15min.

Una vez finalizada la agregación, se generan las variables temporales año, mes, día, hora, día_semana y es_fin_semana. Posteriormente, los registros horarios se relacionan mediante el identificador id con la información de los puntos de medida, incorporando al conjunto procesado los metadatos geográficos y descriptivos disponibles.

Control o transformación	Tratamiento implementado
Archivos históricos mensuales	Lectura secuencial de los CSV disponibles
Volumen de los archivos	Lectura por bloques de 500.000 registros

Registros señalados con error	Conservación de registros con error = N
Campo temporal	Conversión de fecha a formato fecha-hora
Variables numéricas	Conversión explícita a formato numérico
Registros sin datos esenciales	Eliminación cuando falta id, fecha o intensidad
Intensidades negativas	Exclusión
Duplicados punto-fecha	Interrupción de la ejecución para su supervisión
Granularidad original	Agrupación por punto de medida y hora
Combinaciones punto-hora incompletas	Exclusión cuando el grupo no contiene cuatro registros
Intensidad horaria	Media de las observaciones del grupo
Variables operativas	Media de ocupación, carga, velocidad y periodo de integración
Información geográfica	Unión con los metadatos mediante id

Tabla 9. Principales controles y transformaciones aplicados durante la preparación de los datos. Elaboración propia.

5.8.Construcción del dataset analítico

El proceso de ingesta y limpieza genera un conjunto horario en el que cada fila corresponde a una combinación de punto de medida y hora que ha superado los controles descritos anteriormente. Este conjunto se almacena en `trafico_limpio.csv` e incorpora variables temporales, indicadores de tráfico y metadatos geográficos.

Para construir el conjunto utilizado por el modelo predictivo se realiza una segunda etapa de preparación. Los registros se ordenan de forma independiente para cada punto de medida y

se genera un calendario horario comprendido entre la primera y la última observación disponible de cada sensor. Las horas para las que no existe una observación permanecen como valores ausentes, evitando sustituirlas artificialmente por datos correspondientes a otros instantes.

Sobre cada serie temporal se generan variables históricas utilizando únicamente observaciones anteriores. En concreto, se calculan la intensidad de la hora anterior (`intensidad_lag_1h`), la intensidad correspondiente a 24 horas antes (`intensidad_lag_24h`) y medias móviles de intensidad de 3 y 24 horas. Para las medias móviles, la serie se desplaza previamente una hora antes de aplicar la ventana de cálculo, de manera que la intensidad correspondiente a la hora objetivo no interviene en la construcción de estas variables.

También se generan versiones retardadas una hora de las variables operativas disponibles: ocupación media, carga media, velocidad media y número de registros utilizados en la agregación. Finalmente, se incorporan las variables temporales necesarias para el modelado y se eliminan de este conjunto las filas que no disponen de todas las variables predictoras requeridas o de la variable objetivo.

Como resultado se obtiene un conjunto específicamente preparado para el modelado predictivo. Por tanto, debe distinguirse entre el conjunto horario generado durante la limpieza y el dataset utilizado para entrenar y evaluar el modelo: el primero conserva las observaciones horarias que cumplen los controles de preparación, mientras que el segundo requiere además disponer de la información histórica necesaria para construir todas las variables predictoras.

Grupo	Variables principales	Origen
Identificación	id, nombre, distrito	Ingesta y metadatos
Temporales	fecha_hora, año, mes, dia, hora, dia_semana, es_fin_semana	Preparación temporal
Tráfico horario	intensidad_horaria, ocupacion_media, carga_media, velocidad_media, registros_15min	Agregación horaria

Geográficas	latitud, longitud	Metadatos de puntos de medida
Históricas de intensidad	Intensidad_lag_1h, intensidad_lag_24h, media_movil_3h, media_movil_24h	Preparación para modelado
Operativas retardadas	Ocupación, carga, velocidad y registros_15min desplazados una hora	Preparación para modelado

Tabla 10. Principales grupos de variables utilizados en la preparación del conjunto analítico. Elaboración propia.

5.9. Modelado predictivo de intensidad del tráfico

El problema predictivo se formula como una tarea de regresión supervisada, en la que la variable objetivo es `intensidad_horaria`. El objetivo consiste en estimar la intensidad correspondiente a una hora determinada y a un punto de medida concreto a partir de información temporal e histórica disponible previamente.

Las variables explicativas incluyen el identificador del punto de medida, características temporales, retardos de intensidad, medias móviles y variables operativas desplazadas temporalmente. Entre las variables históricas se encuentran `intensidad_lag_1h` e `intensidad_lag_24h`, junto con las medias móviles de intensidad correspondientes a las 3 y 24 horas anteriores. Para las variables operativas de ocupación, carga, velocidad y número de registros se utilizan valores retardados una hora.

La construcción temporal de estas variables tiene como finalidad evitar que el modelo utilice información de la propia hora objetivo que no estaría disponible previamente. Este criterio resulta especialmente relevante en problemas predictivos temporales, ya que la utilización durante el entrenamiento de información no disponible en el momento de realizar la predicción puede producir estimaciones excesivamente optimistas del rendimiento del modelo.

El algoritmo seleccionado es `RandomForestRegressor`, implementado mediante `scikit-learn`. El modelo utiliza 100 árboles, una profundidad máxima de 14, un mínimo de cinco observaciones

por hoja y `random_state=42`. Además, se establece `n_jobs=-1` para utilizar los procesadores disponibles durante el entrenamiento.

5.10. Estrategia de entrenamiento y evaluación

La evaluación del modelo se ha diseñado mediante una partición cronológica de los datos. Este criterio evita utilizar observaciones futuras para entrenar un modelo que posteriormente se evalúa sobre observaciones anteriores. La documentación de scikit-learn recomienda considerar explícitamente el orden de las observaciones en la evaluación de problemas temporales, debido a la dependencia existente entre observaciones próximas en el tiempo.

En la primera etapa se establecen tres periodos: entrenamiento inicial desde el comienzo de 2023 hasta el 30 de septiembre de 2023, validación desde el 1 de octubre hasta el 31 de diciembre de 2023 y prueba desde el 1 de enero de 2024. La separación se implementa directamente mediante las marcas temporales de cada observación.

El modelo se entrena inicialmente con el primer periodo y se evalúa sobre el conjunto de validación. Una vez completada esta fase, los conjuntos de entrenamiento y validación se combinan para realizar el entrenamiento final con los datos elegibles de 2023. El modelo resultante se evalúa sobre el conjunto de prueba correspondiente a 2024, que contiene 371.128 observaciones.

Como referencia se utilizan dos predictores temporales sencillos. El primero, denominado persistencia, utiliza como predicción la intensidad observada una hora antes (`intensidad_lag_1h`). El segundo utiliza la intensidad correspondiente a 24 horas antes (`intensidad_lag_24h`). Ambos se evalúan sobre las mismas observaciones utilizadas para evaluar Random Forest.

El rendimiento se cuantifica mediante el error absoluto medio (MAE), la raíz del error cuadrático medio (RMSE) y el coeficiente de determinación R^2 . El MAE representa el promedio de los errores absolutos y toma como valor óptimo cero. El RMSE se obtiene a partir de los errores elevados al cuadrado, por lo que presenta una mayor sensibilidad a errores de gran magnitud. R^2 expresa el rendimiento del modelo en relación con una predicción de referencia basada en la media de la variable objetivo y no debe interpretarse como un porcentaje de predicciones correctas.

Modelo	MAE (vehículos/h)	RMSE (vehículos/h)	R ²
Persistencia (lag 1h)	88,14	150,49	0,8764
Baseline lag 24h	105,81	205,95	0,7685
Random Forest Regressor	52,48	89,29	0,9565

Tabla 11. Métricas de evaluación del modelo predictivo. Elaboración propia.

Los resultados obtenidos muestran que Random Forest presenta el mejor rendimiento de los tres modelos evaluados sobre el conjunto de prueba de 2024. El modelo obtiene un MAE de 52,48 vehículos/hora, frente a 88,14 vehículos/hora del modelo de persistencia y 105,81 vehículos/hora del baseline de 24 horas. Esto supone una reducción del MAE de aproximadamente el 40,5 % y el 50,4 %, respectivamente.

La misma tendencia se observa en el RMSE, que alcanza 89,29 vehículos/hora para Random Forest, frente a 150,49 en persistencia y 205,95 en el baseline de 24 horas. El coeficiente R² del modelo alcanza 0,9565, superior a los valores de 0,8764 y 0,7685 obtenidos por los dos modelos de referencia. En conjunto, las tres métricas indican que Random Forest proporciona una mejora sobre ambas estrategias temporales de referencia en las observaciones utilizadas como conjunto de prueba.

5.11. Importancia de las variables

Una vez entrenado el modelo final, se obtiene la importancia de las variables proporcionada por RandomForestRegressor. Los resultados muestran un predominio de las variables que contienen información histórica de intensidad, especialmente la correspondiente a la hora inmediatamente anterior.

Variable	Importancia aproximada
intensidad_lag_1h	86,09%
intensidad_lag_24h	8,65%

hora	1,75%
Media_movil_3h	1,11%
Resto de variables	<3% conjuntamente

Tabla 12. Variables con mayor importancia en el modelo Random Forest. Elaboración propia.

La variable intensidad_lag_1h concentra aproximadamente el 86,09 % de la importancia calculada por el modelo, seguida de intensidad_lag_24h, con aproximadamente un 8,65 %. La variable hora y la media móvil de las tres horas anteriores presentan una contribución considerablemente menor.

Esta distribución muestra que las predicciones realizadas por el modelo dependen principalmente de la información reciente de intensidad disponible para cada punto de medida. La importancia asociada a intensidad_lag_24h indica también que la información correspondiente al mismo intervalo del día anterior aporta capacidad predictiva al modelo.

Estas importancias deben interpretarse como una descripción del funcionamiento interno del Random Forest entrenado y no como evidencia de relaciones causales entre las variables y la intensidad del tráfico.

La variable retardada correspondiente a registros_15min presenta una importancia nula en el modelo. Este resultado es coherente con el proceso de preparación, ya que las observaciones horarias conservadas contienen cuatro registros y, por tanto, esta variable no aporta variabilidad entre las observaciones utilizadas.

5.12. Análisis complementario de los errores

Además de las métricas globales, se ha analizado la distribución de los errores obtenidos por Random Forest con el objetivo de caracterizar con mayor detalle su comportamiento sobre el conjunto de prueba.

El error absoluto mediano es de aproximadamente 28,06 vehículos/hora, mientras que los percentiles 90 y 95 alcanzan 128,95 y 181,44 vehículos/hora, respectivamente. La diferencia entre la mediana y los percentiles superiores muestra que una parte de las observaciones presenta errores considerablemente mayores que el comportamiento habitual del modelo.

El error medio firmado, calculado como la diferencia entre la intensidad real y la predicha, es de aproximadamente 0,07 vehículos/hora. Este valor, próximo a cero, indica que las desviaciones positivas y negativas se compensan globalmente en el conjunto analizado. No obstante, esta medida agregada no permite descartar la existencia de sesgos en periodos o puntos de medida concretos.

El análisis temporal muestra que el MAE de Random Forest es inferior al obtenido por el modelo de persistencia en los meses analizados de 2024. La cobertura de datos no es, sin embargo, uniforme durante todo el periodo. En particular, octubre y noviembre presentan una menor disponibilidad de observaciones elegibles, por lo que los resultados correspondientes a estos meses se interpretan únicamente sobre los datos disponibles y no como evidencia de un mejor funcionamiento del modelo durante dichos periodos.

También se observan diferencias entre puntos de medida. El MAE mediano calculado por punto es de aproximadamente 48,38 vehículos/hora, aunque determinados sensores presentan errores superiores. Esta dimensión se incorpora al informe interactivo para permitir el análisis de la distribución espacial de los errores.

5.13. Diseño del informe interactivo en Power BI

La capa de visualización se ha implementado mediante un informe interactivo en Power BI. Los datos utilizados por esta capa se preparan mediante el script `03_export_powerbi.py`, que genera archivos específicos para el análisis de los datos de tráfico, la dimensión temporal, los puntos de medida, las predicciones, las métricas del modelo, la importancia de las variables y los indicadores relacionados con la preparación y calidad de los datos.

La interacción con la información se realiza mediante los mecanismos disponibles en las páginas del informe, entre ellos filtros y segmentadores. En Power BI, los segmentadores constituyen controles de filtrado interactivos situados directamente en el lienzo y permiten restringir los datos representados en otros objetos visuales.

El informe desarrollado se organiza en ocho páginas:

1. Portada, que presenta el proyecto y proporciona acceso a las distintas secciones del informe.

2. Visión general, destinada a resumir los principales indicadores y la evolución global de los datos.
3. Calidad de datos, que presenta información relacionada con el procesamiento, la cobertura y los controles aplicados a los datos.
4. Análisis temporal, orientada al estudio de la evolución de la intensidad según diferentes dimensiones temporales.
5. Análisis geográfico, que representa los puntos de medida y permite analizar su comportamiento espacial.
6. Modelo predictivo, destinada a comparar los valores reales y predichos y a presentar las métricas obtenidas por los modelos evaluados.
7. Análisis de errores, que permite examinar la magnitud y distribución de los errores de predicción.
8. Arquitectura y metodología, que resume los principales componentes del pipeline y el procedimiento seguido durante el desarrollo.

La separación en páginas permite diferenciar el análisis de los datos originales y procesados, la evaluación del modelo predictivo y la descripción de la solución desarrollada. En particular, las páginas de calidad de datos y análisis de errores complementan las métricas globales mediante información sobre la cobertura del conjunto procesado y la distribución de los errores de predicción.

5.14. Validación de la solución

5.14.1. Pruebas funcionales

Prueba	Funcionalidad evaluada	Procedimiento	Criterio de aceptación
PF01	Inspección de datos	Ejecutar <code>00_inspeccion_datos.py</code> sobre los archivos originales	Los archivos son detectados y se muestran correctamente

			columnas y registros de muestra.
PF02	Ingesta y limpieza	Ejecutar 01_ingesta_limpieza.py	El proceso finaliza y genera los archivos de salida esperados
PF03	Control de duplicados	Comprobar el control implementado sobre id y fecha	El pipeline detecta duplicados cuando existen
PF04	Agregación horaria	Revisar las observaciones generadas	Las horas conservadas contienen cuatro registros según el criterio implementado
PF05	Construcción del dataset predictivo	Ejecutar 02_modelo_predictivo.py	Se generan variables históricas y las particiones temporales esperadas
PF06	Evaluación del modelo	Comprobar metricas_modelo.csv	Se generan métricas para persistencia, baseline 24 h y Random Forest
PF07	Exportación a Power BI	Ejecutar 03_export_powerbi.py	Se generan los diez CSV previstos para Power BI
PF08	Informe Power BI	Abrir el .pbix y recorrer sus páginas	Las ocho páginas se cargan y permiten consultar las visualizaciones previstas

PF09	Interacción	Aplicar filtros/segmentadores en Power BI	Los elementos visuales afectados responden a la selección realizada
PF10	Navegación	Utilizar los controles de navegación del informe	Es posible acceder a las páginas previstas mediante los elementos implementados

Tabla 13. Validación de la solución. Elaboración propia.

Funcionalidad evaluada	Resultado obtenido	Estado
PF01	Se localizaron 24 archivos históricos de tráfico y 1 fichero Excel de puntos de medida. Los CSV se leyeron correctamente con separador ; y codificación UTF-8, y el fichero Excel se procesó sin errores. El script mostró las columnas y las primeras observaciones de los archivos inspeccionados.	Superada
PF02	Se procesaron los 24 archivos históricos de 2023–2024, se depuraron 4.962 puntos de medida y se seleccionaron 50 puntos para el caso de estudio. El pipeline finalizó sin errores y generó un dataset consolidado de 833.086 observaciones horarias en datos/procesados/definitivos.	Superada
PF03	El procesamiento partió de 308.458.911 registros, conservó 3.339.435 registros válidos tras la limpieza y excluyó 3.142 combinaciones punto-hora incompletas. El resultado consolidado contiene 833.086 observaciones horarias.	Superada
PF04	El archivo contiene 833.086 observaciones, 19 variables y 50 puntos de medida, con fechas comprendidas entre enero de	Superada

	2023 y diciembre de 2024. No se detectaron duplicados id-fecha_hora ni intensidades horarias negativas, y todas las observaciones conservadas presentan cuatro registros en registros_15min.	
PF05	El proceso utilizó 788.913 observaciones disponibles para modelado, distribuidas en 311.454 de entrenamiento, 106.331 de validación y 371.128 de prueba. Se ejecutaron la persistencia, el baseline de 24 horas y Random Forest, y el proceso finalizó correctamente generando los archivos de resultados.	Superada
PF06	El archivo contiene resultados para los tres modelos sobre 371.128 observaciones de prueba. Random Forest obtuvo MAE = 52,48 veh/h, RMSE = 89,29 veh/h y $R^2 = 0,9565$, frente a MAE = 88,14, RMSE = 150,49 y $R^2 = 0,8764$ de persistencia, y MAE = 105,81, RMSE = 205,95 y $R^2 = 0,7685$ del baseline de 24 horas.	Superada
PF07	El script finalizó correctamente y generó 10 archivos CSV en datos/procesados/definitivos/powerbi, correspondientes a tráfico, análisis temporal y por puntos, predicciones, métricas, importancia de variables, calidad, limpieza, resumen general y puntos de medida.	Superada
PF08	El informe contiene 8 páginas: Portada, Visión general, Calidad de datos, Análisis temporal, Análisis geográfico, Modelo predictivo, Análisis de errores y Arquitectura y metodología.	Superada
PF09	Al seleccionar 2023 y 2024 se actualizaron las tarjetas de intensidad media y horas disponibles, así como los gráficos de intensidad por mes, intensidad por hora y distribución por día	Superada

	de la semana. Por ejemplo, la intensidad media pasó de 452,81 en 2023 a 450,23 en 2024.	
PF10	Desde la Portada se seleccionó la opción Visión general y el informe mostró correctamente dicha página, quedando además resaltada la opción activa en el menú de navegación.	Superada

Tabla 14. Validación de la solución. Elaboración propia.

La prueba PF01 verificó la capacidad del script `00_inspeccion_datos.py` para localizar e inspeccionar los datos originales utilizados por la solución. Durante la ejecución se detectaron los 24 archivos mensuales de tráfico correspondientes al periodo 2023–2024 y el fichero de puntos de medida. Los archivos CSV se leyeron correctamente con separador ; y codificación UTF-8, mientras que el fichero geográfico en formato Excel se procesó sin errores. En ambos casos, el script identificó las columnas disponibles y mostró una muestra de las observaciones contenidas en los archivos. La prueba se consideró superada.

La prueba PF02 verificó el funcionamiento completo del proceso de ingesta, limpieza y agregación horaria. El script `01_ingesta_limpieza.py` detectó y procesó los 24 archivos históricos correspondientes a 2023 y 2024, junto con el fichero de puntos de medida. Durante la ejecución se depuraron 4.962 puntos de medida y se seleccionaron 50 para el caso de estudio. Cada archivo mensual fue procesado de forma independiente, registrando el número de observaciones leídas, válidas y agregadas por hora. Tras integrar los resultados mensuales, el pipeline generó un conjunto final de 833.086 observaciones horarias y finalizó sin errores, por lo que la prueba se consideró superada.

La prueba PF03 evaluó la aplicación de los controles de calidad incorporados al proceso de preparación de los datos. El fichero `resumen_limpieza.csv`, generado automáticamente durante la ejecución, permitió comprobar los resultados obtenidos para cada uno de los 24 archivos mensuales. En conjunto, el proceso partió de 308.458.911 registros y conservó 3.339.435 registros válidos correspondientes a los 50 puntos seleccionados tras aplicar las condiciones de limpieza. Durante la agregación se excluyeron 3.142 combinaciones punto-hora que no cumplían la condición de completitud establecida. El proceso produjo finalmente 833.086 observaciones horarias. La prueba se consideró superada.

La prueba PF04 comprobó la consistencia del conjunto horario generado por el proceso de preparación. El archivo `trafico_limpio.csv` contiene 833.086 observaciones y 19 variables correspondientes a 50 puntos de medida, con información comprendida entre enero de 2023 y diciembre de 2024. No se detectaron duplicados para una misma combinación de punto de medida y hora ni valores negativos en la variable `intensidad_horaria`. Asimismo, las 833.086 observaciones conservadas presentan un valor igual a cuatro en `registros_15min`, de acuerdo con la condición de completitud aplicada durante la agregación. Se identificaron 4.296 valores ausentes en `ocupacion_media`, variable que no interviene como predictor en el modelo final. A partir de estas comprobaciones, la prueba se consideró superada.

La prueba PF05 verificó la ejecución del proceso de modelado predictivo mediante el script `02_modelo_predictivo.py`. A partir de las 833.086 observaciones del conjunto horario, la construcción de las variables temporales e históricas permitió disponer de 788.913 observaciones elegibles para el modelado. La división temporal utilizada comprendió 311.454 observaciones de entrenamiento, 106.331 de validación y 371.128 de prueba. El script ejecutó los dos modelos de referencia y el modelo Random Forest Regressor y finalizó correctamente, almacenando los resultados en el directorio de datos procesados. Por tanto, la prueba se consideró superada.

La prueba PF06 comprobó el fichero de métricas generado por el proceso de modelado. Los tres modelos fueron evaluados sobre el mismo conjunto de prueba, formado por 371.128 observaciones. Random Forest Regressor obtuvo un MAE de 52,48 vehículos/hora, un RMSE de 89,29 vehículos/hora y un R^2 de 0,9565. El modelo de persistencia obtuvo valores de 88,14, 150,49 y 0,8764, respectivamente, mientras que el modelo basado en la intensidad de 24 horas anteriores alcanzó 105,81, 205,95 y 0,7685. La correspondencia entre las métricas almacenadas y las mostradas durante la ejecución permitió considerar superada la prueba.

La prueba PF07 verificó la preparación de los datos utilizados por la capa de visualización. La ejecución de `03_export_powerbi.py` finalizó sin errores y generó diez archivos CSV en el directorio destinado a Power BI. Las salidas contienen conjuntos diferenciados para el análisis del tráfico, la dimensión temporal y geográfica, las predicciones, las métricas del modelo, la importancia de variables y los indicadores de calidad y limpieza. La presencia de los diez archivos previstos permitió considerar superada la prueba.

La prueba PF08 comprobó la estructura del informe desarrollado en Power BI. El archivo `dashboard_trafico.pbix` contiene ocho páginas organizadas en Portada, Visión general, Calidad de datos, Análisis temporal, Análisis geográfico, Modelo predictivo, Análisis de errores y Arquitectura y metodología. Esta estructura permite separar la presentación general de la solución, el análisis de los datos, la evaluación del modelo predictivo y la documentación de la arquitectura desarrollada. La prueba se consideró superada.

La prueba PF09 evaluó la interactividad del informe mediante el segmentador temporal disponible en la página de visión general. Se realizaron dos consultas seleccionando alternativamente los años 2023 y 2024. Al modificar la selección, las tarjetas y los gráficos relacionados se actualizaron de acuerdo con el periodo elegido. Entre otros cambios, la intensidad media mostrada pasó de 452,81 para 2023 a 450,23 para 2024, mientras que las horas disponibles pasaron de aproximadamente 434 mil a 399 mil. También se actualizaron los gráficos de intensidad por mes, intensidad por hora y distribución por día de la semana. La respuesta conjunta de los elementos visuales permitió considerar superada la prueba.

La prueba PF10 evaluó el funcionamiento de la navegación incorporada al informe. Desde la Portada se utilizó el menú superior para acceder a la página Visión general. La acción permitió mostrar correctamente la página seleccionada y el menú identificó visualmente la sección activa. La prueba confirmó el funcionamiento de la navegación implementada y se consideró superada.

5.14.2. Prueba de aceptación y usabilidad

Además de las pruebas funcionales, se realizó una prueba exploratoria de aceptación y usabilidad del informe desarrollado en Power BI. En la evaluación participaron cuatro personas con distintos niveles de experiencia previa en herramientas de análisis y visualización de datos. Dos participantes indicaron disponer de experiencia básica, uno de experiencia intermedia y uno no tenía experiencia previa. Dos de los participantes habían utilizado Power BI anteriormente y los otros dos no.

Antes de completar el cuestionario, los participantes realizaron una serie de tareas sobre el informe: navegar desde la portada hasta las distintas páginas, utilizar los filtros temporales, consultar la información relativa a la calidad de los datos, identificar los resultados del modelo predictivo y acceder al análisis de errores y a la descripción de la arquitectura.

Tras realizar estas tareas, se solicitó valorar ocho afirmaciones mediante una escala de 1 a 5, donde 1 correspondía a “totalmente en desacuerdo” y 5 a “totalmente de acuerdo”. Los aspectos evaluados fueron la claridad de la navegación, la facilidad de utilización de los filtros, la localización de los indicadores, la comprensión de la comparación entre modelos, la presentación de la calidad de los datos, la interpretación del análisis de errores, la organización de las páginas y la utilidad general del informe. El cuestionario incluyó además dos preguntas abiertas destinadas a identificar dificultades y posibles mejoras.

Las valoraciones obtenidas fueron positivas en todos los aspectos evaluados. La puntuación media más elevada correspondió a la comprensión de la comparación entre los modelos predictivos, con 4,75 sobre 5. La localización de los principales indicadores obtuvo una media de 4,67 sobre 5, calculada a partir de tres respuestas válidas, al no haberse respondido este ítem en uno de los cuestionarios. La navegación, el análisis de errores y la organización de las páginas alcanzaron una valoración media de 4,50. La facilidad de uso de los filtros, la comprensión de la información sobre calidad de los datos y la utilidad general obtuvieron una media de 4,25. Considerando las respuestas disponibles a los ocho ítems, la valoración media global fue de 4,45 sobre 5.

Las respuestas abiertas permitieron identificar algunos aspectos susceptibles de mejora. Aunque dos participantes indicaron que el contenido resultaba claro o que el informe estaba bien planteado, también se señalaron dificultades relacionadas con la navegación y con la interpretación de los filtros. Entre las propuestas de mejora aparecieron la incorporación de una mayor variedad de filtros, la revisión de los colores y una mayor variedad de tipos de visualización. Estas observaciones se consideran posibles mejoras de la interfaz y no impidieron completar las tareas planteadas durante la prueba.

5.14.3. Resultados de la validación

La validación de la solución combinó pruebas funcionales del pipeline de datos y del informe interactivo con una prueba exploratoria de aceptación y usabilidad. Las diez pruebas funcionales definidas fueron ejecutadas y superadas. Estas pruebas permitieron comprobar la inspección de los datos originales, el proceso de ingesta y limpieza, los controles de calidad, la generación del dataset horario, la ejecución y evaluación del modelo predictivo, la preparación de los conjuntos destinados a Power BI y el funcionamiento de la estructura, los filtros y la navegación del informe.

La ejecución del pipeline permitió reproducir los principales resultados cuantitativos documentados en el trabajo. El conjunto procesado quedó formado por 833.086 observaciones horarias correspondientes a 50 puntos de medida. El proceso de modelado generó 371.128 observaciones para el conjunto final de prueba y reprodujo para Random Forest un MAE de 52,48 vehículos/hora, un RMSE de 89,29 vehículos/hora y un R^2 de 0,9565. Asimismo, se comprobó la generación de los diez conjuntos de datos utilizados por Power BI y el funcionamiento de las ocho páginas que forman el informe interactivo.

La prueba exploratoria realizada con cuatro participantes obtuvo una valoración media global de 4,45 sobre 5 en los ítems respondidos. Las puntuaciones medias de todos los aspectos evaluados se situaron entre 4,25 y 4,75. Los comentarios abiertos mostraron una valoración favorable del informe, aunque también identificaron posibilidades de mejora relacionadas principalmente con los filtros, la navegación, los colores y la variedad de las visualizaciones.

En conjunto, las evidencias obtenidas permiten comprobar el funcionamiento de los principales componentes desarrollados dentro del alcance definido para el proyecto. La validación de usuarios debe interpretarse como una evaluación exploratoria debido al reducido número de participantes y no como un estudio representativo de usabilidad.

5.15. Reutilización y posibilidades de ampliación

La arquitectura desarrollada separa las principales responsabilidades del proceso en componentes diferenciados de adquisición, preparación de datos, modelado predictivo y visualización. Esta organización permite identificar las entradas, transformaciones y salidas correspondientes a cada etapa y facilita la modificación independiente de determinados componentes.

La implementación y evaluación realizadas en este TFM se limitan al caso de estudio basado en datos históricos de tráfico de Madrid y al entorno local utilizado durante el desarrollo. Por este motivo, el trabajo no demuestra experimentalmente la reutilización de la arquitectura en otros dominios ni su comportamiento ante volúmenes o condiciones diferentes de los evaluados.

La adaptación a otro ámbito requeriría, entre otros cambios, sustituir o modificar las fuentes de datos, definir los controles de calidad correspondientes, establecer una nueva variable

objetivo y sus variables predictoras y adaptar la capa de visualización a los indicadores relevantes del nuevo contexto. En consecuencia, la modularidad de la solución proporciona una estructura susceptible de adaptación, pero no implica que pueda trasladarse directamente a otros dominios sin modificaciones y una nueva evaluación.

Del mismo modo, la implementación actual utiliza procesamiento local por lotes y archivos estructurados como mecanismo de intercambio entre las diferentes etapas. Una ampliación orientada a mayores volúmenes de información o a procesamiento continuo requeriría evaluar tecnologías y mecanismos diferentes de los implementados en este trabajo. El procesamiento distribuido, las bases de datos, el almacenamiento en la nube y el procesamiento de flujos constituyen posibles líneas de evolución, pero no forman parte de las capacidades implementadas ni evaluadas en el presente TFM.

6. Código fuente y datos analizados

Este capítulo documenta la organización del código fuente y de los datos utilizados durante la implementación. Su finalidad es complementar el desarrollo técnico descrito en el capítulo anterior y proporcionar la información necesaria para identificar los componentes que forman la solución y la secuencia seguida para su ejecución.

El proyecto mantiene separados el código fuente, los datos originales, los datos procesados y los archivos preparados para la capa de visualización. Esta organización permite diferenciar las entradas originales de los resultados generados durante las sucesivas etapas del procesamiento y evita modificar directamente los archivos descargados de las fuentes de datos.

El código fuente, las dependencias y la documentación asociada se organizan en un repositorio que permite identificar los scripts desarrollados y el orden en que intervienen dentro del pipeline. Los datos originales y los conjuntos procesados se documentan de forma diferenciada para mantener la correspondencia entre las fuentes utilizadas y los resultados obtenidos.

6.1. Código fuente y organización del repositorio

El código desarrollado se organiza en una estructura de directorios que separa los scripts de ejecución, los datos originales, los conjuntos procesados y los archivos destinados a la visualización. Esta separación permite identificar las entradas y salidas de las diferentes etapas del pipeline y mantener los datos originales sin modificaciones.

Los scripts utilizan rutas predefinidas dentro de esta estructura para localizar sus entradas y almacenar los resultados generados. De esta forma, las sucesivas fases del procesamiento mantienen una organización común y pueden ejecutarse siguiendo el orden establecido en la documentación del proyecto.

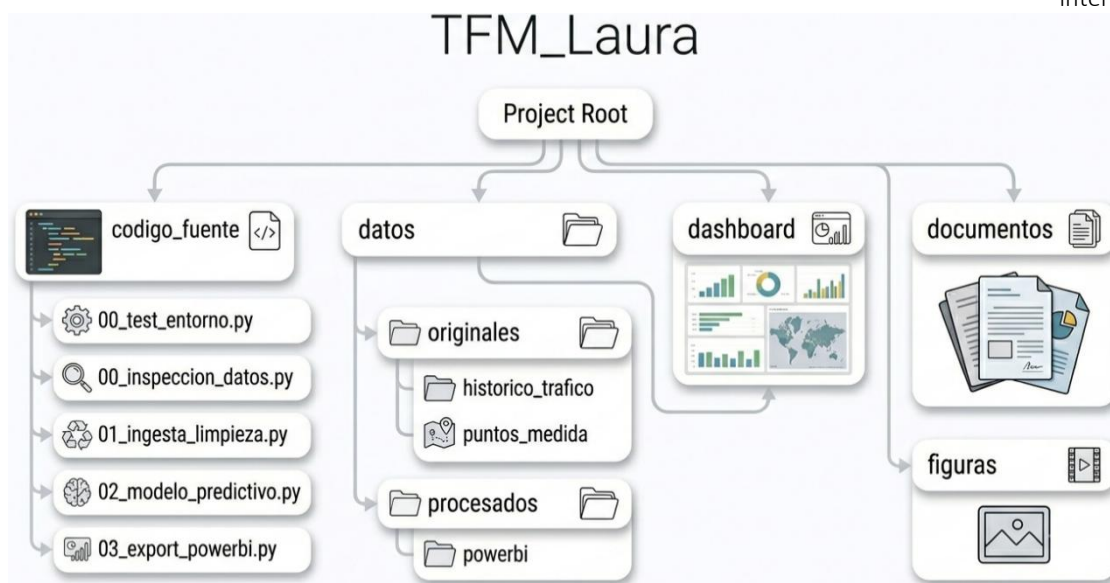


Figura 3. Organización del repositorio donde se encuentra el proyecto. Elaboración propia.

La carpeta de datos originales se mantiene separada de la carpeta de resultados procesados. De esta forma, los archivos descargados de las fuentes oficiales permanecen sin modificar, mientras que las transformaciones sucesivas se almacenan en directorios específicos.

La utilización de una estructura de directorios estable también facilita que los scripts puedan localizar automáticamente sus entradas y escribir sus salidas en rutas predefinidas, reduciendo la necesidad de modificar manualmente las rutas durante cada ejecución.

6.1.1. Scripts desarrollados

Script	Función
00_test_entorno.py	Comprueba la disponibilidad de las bibliotecas principales utilizadas por el proyecto.
00_inspeccion_datos.py	Inspecciona la estructura de los archivos originales, incluyendo columnas y primeras observaciones.
01_ingesta_limpieza.py	Lee los archivos históricos, selecciona los puntos de medida, aplica los controles de limpieza, realiza la agregación horaria e incorpora los metadatos geográficos.

02_modelo_predictivo.py	Construye las variables predictoras, realiza la partición temporal, entrena Random Forest, evalúa los modelos de referencia y exporta predicciones y métricas.
03_export_powerbi.py	Genera los conjuntos de datos utilizados posteriormente por el informe de Power BI.

Tabla 16. Scripts desarrollados durante el proyecto. Elaboración propia

El script 01_ingesta_limpieza.py concentra las principales operaciones de preparación de los datos. Entre otras funciones, realiza la selección de los puntos incluidos en el caso de estudio, aplica controles sobre los registros originales, comprueba la existencia de duplicados, construye las observaciones horarias e incorpora la información descriptiva y geográfica de los puntos de medida.

El script 02_modelo_predictivo.py prepara las series temporales por punto de medida y genera las variables históricas utilizadas durante el modelado. Los datos se dividen cronológicamente en entrenamiento, validación y prueba. Posteriormente, se evalúan dos modelos temporales de referencia y el modelo Random Forest Regressor, generándose los archivos de predicciones, métricas e importancia de variables.

Finalmente, 03_export_powerbi.py transforma los resultados de las etapas anteriores en conjuntos específicos destinados a la capa de visualización. Entre las salidas se incluyen datos de tráfico, agregaciones temporales, información de puntos de medida, predicciones, métricas, importancia de variables e indicadores relacionados con la preparación y calidad de los datos.

6.1.2. Documentación y dependencias

El repositorio incluye un archivo requirements.txt destinado a especificar las dependencias necesarias para ejecutar los scripts del proyecto. Este archivo documenta las bibliotecas requeridas por la implementación y facilita la preparación de un entorno compatible con el código desarrollado.

La documentación se complementa mediante el archivo README.md, en el que se describe la estructura general del proyecto y la secuencia de ejecución de los scripts. La ejecución del

pipeline sigue el orden establecido por los componentes de inspección, ingesta y limpieza, modelado predictivo y preparación de los datos para Power BI.

Esta documentación permite identificar las dependencias, los archivos de entrada y las principales etapas necesarias para reproducir el procesamiento realizado en el proyecto.

6.2. Datos analizados

6.2.1. Datos originales utilizados

Los datos originales utilizados por la solución se almacenan bajo el directorio datos/originales y se mantienen separados de los conjuntos generados durante el procesamiento. Esta organización conserva una referencia de los archivos descargados de las fuentes utilizadas y evita que las sucesivas transformaciones modifiquen directamente los datos de entrada.

La subcarpeta historico_trafico contiene los 24 archivos mensuales correspondientes al periodo comprendido entre enero de 2023 y diciembre de 2024. Estos archivos constituyen la entrada principal del proceso de ingesta y contienen las observaciones históricas utilizadas para construir el conjunto horario.

La subcarpeta puntos_medida contiene el archivo utilizado para incorporar información descriptiva y geográfica de los puntos de medida. El identificador id permite relacionar esta información con los registros de tráfico e incorporar al conjunto procesado los campos disponibles utilizados por la implementación, entre ellos el distrito, el nombre del punto, la longitud y la latitud.

Ambos conjuntos proceden del Portal de Datos Abiertos del Ayuntamiento de Madrid. Su procedencia, características y función dentro del proyecto se han descrito en el capítulo 5, mientras que las condiciones aplicables a su reutilización se recogen en el marco normativo del capítulo 4.

6.2.2. Datos procesados generados

La ejecución del pipeline genera diferentes archivos destinados a conservar los resultados de las etapas de limpieza, preparación analítica, modelado predictivo y visualización. Los archivos se almacenan en los directorios correspondientes a los datos procesados y constituyen las entradas utilizadas por las etapas posteriores de la solución.

La principal salida de la fase de ingesta y limpieza es `trafico_limpio.csv`, que contiene 833.086 observaciones horarias correspondientes a los 50 puntos de medida seleccionados durante el periodo 2023–2024. A partir de este conjunto se generan posteriormente las variables necesarias para el modelado y los archivos que contienen las predicciones, métricas y resultados auxiliares.

Archivo generado	Contenido principal
<code>trafico_limpio.csv</code>	Conjunto horario procesado con 833.086 observaciones.
<code>resumen_limpieza.csv</code>	Resumen del procesamiento de los archivos históricos y de las observaciones generadas.
<code>puntos_medida_limpios.csv</code>	Información depurada de los puntos de medida utilizada durante la integración de metadatos.
<code>puntos_seleccionados_modelo.csv</code>	Identificadores de los 50 puntos seleccionados para el caso de estudio.
<code>trafico_modelo.csv</code>	Conjunto analítico con las variables utilizadas durante el modelado predictivo.
<code>predicciones.csv</code>	Valores reales, predicciones de los modelos evaluados y errores de predicción.
<code>metricas_modelo.csv</code>	MAE, RMSE, R^2 y número de observaciones de los modelos evaluados sobre el conjunto de prueba.
<code>importancia_variables.csv</code>	Importancia de las variables obtenidas mediante el modelo Random Forest.
<code>resumen_modelo.csv</code>	Resumen de la distribución temporal de las observaciones utilizadas durante el modelado.

Tabla 17. Principales archivos procesados generados por la solución. Elaboración propia.

El conjunto horario procesado y el conjunto utilizado por el modelo cumplen funciones diferentes. `trafico_limpio.csv` contiene las observaciones horarias que han superado los controles de preparación de datos, mientras que `trafico_modelo.csv` requiere además que cada observación disponga de las variables predictoras necesarias, incluidos los retardos y las medias móviles calculados a partir de información anterior. Las filas que no disponen de toda la información requerida para construir estas variables no se utilizan durante el modelado.

Posteriormente, el dataset analítico se divide cronológicamente en los periodos de entrenamiento, validación y prueba definidos en el capítulo 5. El conjunto de prueba correspondiente a 2024 contiene 371.128 observaciones elegibles, y sobre estas mismas observaciones se calculan las métricas finales de Random Forest y de los dos modelos temporales de referencia.

6.2.3. Archivos destinados a Power BI

El script `03_export_powerbi.py` genera un conjunto de archivos CSV destinados específicamente a la capa de visualización. Estos archivos se construyen a partir de los resultados obtenidos durante las etapas anteriores del pipeline y proporcionan conjuntos diferenciados para los distintos tipos de análisis incluidos en el informe.

Los archivos generados son:

- `trafico_powerbi.csv`
- `trafico_temporal_powerbi.csv`
- `trafico_puntos_powerbi.csv`
- `predicciones_powerbi.csv`
- `metricas_powerbi.csv`
- `importancia_variables_powerbi.csv`
- `resumen_limpieza_powerbi.csv`
- `resumen_general_powerbi.csv`
- `calidad_datos_powerbi.csv`
- `puntos_medida_powerbi.csv`

Los archivos se exportan utilizando una configuración común de separador, codificación y representación decimal, de forma que puedan utilizarse como fuentes de datos de la capa de visualización.

La generación de conjuntos diferenciados responde a las necesidades de las distintas dimensiones de análisis. Los archivos de predicciones, métricas e importancia de variables proporcionan la información utilizada para analizar el comportamiento del modelo predictivo, mientras que los conjuntos relacionados con limpieza y calidad permiten representar información sobre el procesamiento y las características de los datos. De forma complementaria, las tablas temporales y geográficas proporcionan los datos necesarios para analizar la evolución temporal y la distribución de los puntos de medida.

Power BI Desktop admite archivos de texto y CSV como fuentes de datos y permite cargarlos o transformarlos mediante Power Query antes de incorporarlos al modelo utilizado por el informe.

6.3. Disponibilidad del código y los datos

El código fuente desarrollado para el proyecto, junto con la documentación de ejecución y las dependencias necesarias, se encuentra disponible en un repositorio público de GitHub. El repositorio contiene los scripts correspondientes a las etapas de inspección de datos, ingesta y limpieza, modelado predictivo y preparación de los archivos utilizados por Power BI.

Los archivos originales de tráfico y de puntos de medida no se redistribuyen en el repositorio. En su lugar, se documentan las fuentes oficiales del Portal de Datos Abiertos del Ayuntamiento de Madrid utilizadas para su obtención y la organización esperada de estos archivos dentro de la estructura del proyecto. Esta separación permite obtener los datos directamente de su fuente de publicación y ejecutar posteriormente el procesamiento mediante los scripts desarrollados.

[Repositorio del TFM en GitHub](#)

7. Conclusiones

7.1. Conclusiones del proyecto

El desarrollo realizado ha permitido diseñar e implementar una arquitectura modular orientada al procesamiento de datos históricos, el análisis predictivo y la visualización interactiva. La solución se ha aplicado a un caso de estudio de movilidad urbana basado en datos abiertos de tráfico de Madrid, integrando en un mismo flujo las etapas de adquisición, preparación de datos, construcción del conjunto analítico, modelado predictivo y visualización.

El proceso de ingesta se ha aplicado sobre 24 archivos mensuales correspondientes a 2023 y 2024, con un volumen inicial de 308.458.911 registros. Para desarrollar el caso de estudio en el entorno local se seleccionaron 50 puntos de medida. Tras aplicar los controles de preparación y realizar la agregación temporal, se obtuvo un conjunto final de 833.086 observaciones horarias. La organización del procesamiento permite mantener separados los datos originales de los conjuntos generados durante las diferentes etapas del pipeline.

La preparación del conjunto predictivo se ha realizado respetando la disponibilidad temporal de la información. Las variables históricas se construyen a partir de observaciones anteriores a la hora objetivo, mediante retardos de intensidad, medias móviles y variables operativas desplazadas. De este modo, el escenario evaluado utiliza información histórica disponible previamente y mantiene una separación cronológica entre los periodos de entrenamiento, validación y prueba.

En la evaluación final sobre las 371.128 observaciones elegibles de 2024, RandomForestRegressor obtuvo un MAE de 52,48 vehículos/hora, un RMSE de 89,29 vehículos/hora y un R^2 de 0,9565. El modelo de persistencia basado en la intensidad de la hora anterior obtuvo un MAE de 88,14 vehículos/hora, mientras que el baseline correspondiente a 24 horas antes alcanzó 105,81 vehículos/hora. Random Forest reduce, por tanto, el MAE aproximadamente un 40,5 % respecto al primer modelo de referencia y un 50,4 % respecto al segundo.

El análisis de la importancia de las variables muestra que `intensidad_lag_1h` concentra aproximadamente el 86,09 % de la importancia calculada por Random Forest, seguida de

intensidad_lag_24h, con aproximadamente un 8,65 %. Estos resultados muestran que el modelo entrenado se apoya principalmente en la información reciente de intensidad y, en menor medida, en información correspondiente al día anterior. Estas importancias describen el funcionamiento del modelo y no deben interpretarse como relaciones causales.

El análisis complementario de los errores permite observar que el rendimiento no es uniforme en todas las observaciones y puntos de medida. El error absoluto mediano se sitúa aproximadamente en 28,06 vehículos/hora, mientras que determinados sensores presentan errores superiores. Las diferencias de cobertura existentes entre periodos también condicionan la interpretación de las métricas temporales, ya que estas se calculan únicamente sobre las observaciones que disponen de la información necesaria para realizar la predicción.

La capa de visualización se ha materializado mediante un informe interactivo de ocho páginas en Power BI: Portada, Visión general, Calidad de datos, Análisis temporal, Análisis geográfico, Modelo predictivo, Análisis de errores y Arquitectura y metodología. Esta organización permite explorar conjuntamente los datos procesados, su dimensión temporal y geográfica, los resultados predictivos y diferentes indicadores relacionados con la calidad de los datos y los errores del modelo.

En conjunto, el trabajo proporciona una solución software que integra procesamiento de datos, modelado predictivo y visualización dentro de un flujo modular ejecutado en un entorno local. Los resultados obtenidos corresponden específicamente al caso de estudio, al periodo 2023–2024 y a los 50 puntos de medida seleccionados. Por tanto, no se consideran demostradas capacidades de procesamiento distribuido, funcionamiento en tiempo real o aplicación efectiva a otros dominios.

7.2.Relación entre objetivos y resultados obtenidos

Objetivo específico	Resultado obtenido	Grado de cumplimiento
OE1. Definir los requisitos y diseñar la	Se definió una arquitectura modular que organiza las funciones de adquisición, ingesta, preparación, modelado predictivo y visualización, delimitando	Cumplido

arquitectura de la solución.	su implementación a procesamiento local por lotes.	
OE2. Implementar el proceso de preparación de los datos del caso de estudio.	Se procesaron 24 archivos mensuales correspondientes a 2023–2024. A partir de 308.458.911 registros originales y de los 50 puntos seleccionados se obtuvo un conjunto final de 833.086 observaciones horarias.	Cumplido
OE3. Desarrollar y evaluar un modelo para la predicción de la intensidad horaria.	Se construyó un modelo Random Forest y se comparó con dos referencias temporales. Sobre el conjunto de prueba de 2024 obtuvo MAE = 52,48 vehículos/h, RMSE = 89,29 vehículos/h y $R^2 = 0,9565$.	Cumplido
OE4. Diseñar e implementar una capa de visualización interactiva.	Se desarrolló un informe interactivo en Power BI compuesto por ocho páginas para analizar datos, calidad, evolución temporal y geográfica, predicciones y errores.	Cumplido
OE5. Verificar y documentar el funcionamiento de la solución.	Se documentaron la estructura del código, las dependencias, los datos de entrada, los archivos generados y los controles incorporados durante el procesamiento.	Pendiente de cerrar con la validación
OE6. Analizar los resultados y delimitar el alcance de la solución.	Se analizaron las métricas predictivas, la importancia de las variables, la distribución de errores y las limitaciones asociadas al entorno local, la selección de puntos y el alcance experimental.	Cumplido

Tabla 18. Relación entre los objetivos específicos y los resultados obtenidos. Elaboración propia.

7.3. Conclusiones personales y aprendizaje adquirido

Desde una perspectiva académica y técnica, el desarrollo de este TFM me ha permitido consolidar conocimientos relacionados con las diferentes etapas de un proyecto de datos, desde la adquisición y preparación de fuentes abiertas hasta la construcción de un modelo predictivo y la presentación de sus resultados mediante visualización interactiva.

Uno de los principales aprendizajes ha sido comprender la influencia que las decisiones adoptadas durante la preparación de los datos tienen sobre las etapas posteriores del análisis. La definición de una granularidad horaria homogénea, el tratamiento de las observaciones incompletas y la construcción de variables predictoras a partir de información temporal anterior han resultado determinantes para establecer un conjunto analítico coherente con el problema predictivo planteado.

El proyecto también me ha permitido profundizar en el uso de Python y pandas para automatizar tareas de ingesta, limpieza y transformación, así como en scikit-learn para la construcción y evaluación de modelos de regresión. La división cronológica de los datos y la utilización de dos modelos temporales de referencia han permitido comprender la importancia de evaluar un modelo predictivo en relación con alternativas sencillas y bajo un protocolo compatible con la naturaleza temporal de los datos.

En la capa de visualización, el desarrollo del informe en Power BI me ha permitido trabajar en la organización de información procedente de diferentes etapas del pipeline. La incorporación de páginas destinadas a calidad de datos, resultados predictivos y análisis de errores ha permitido presentar conjuntamente los resultados del modelo y el contexto necesario para interpretarlos.

Otro aprendizaje relevante ha sido distinguir entre las capacidades implementadas y aquellas que únicamente constituyen posibilidades de ampliación. El procesamiento por bloques mediante pandas ha permitido ejecutar el caso de estudio en un entorno local con el volumen de información utilizado, pero no permite concluir que la arquitectura presente escalabilidad distribuida. Esta delimitación resulta necesaria para que las conclusiones se correspondan con las capacidades efectivamente evaluadas.

Finalmente, el desarrollo del proyecto me ha permitido integrar programación, tratamiento de datos, aprendizaje automático y visualización dentro de una misma solución. El

seguimiento de las entradas, transformaciones, conjuntos intermedios, predicciones y resultados ha puesto de manifiesto la importancia de mantener la trazabilidad entre los diferentes componentes durante todo el ciclo de procesamiento.

8. Limitaciones y prospectiva

8.1. Limitaciones

Los resultados obtenidos deben interpretarse teniendo en cuenta las limitaciones asociadas al entorno de ejecución, los datos utilizados, el planteamiento predictivo y la capa de visualización.

Una primera limitación corresponde al entorno de ejecución. La arquitectura se ha implementado en un entorno local mediante Python, pandas, archivos CSV, scikit-learn y Power BI Desktop. La lectura por bloques permite procesar los archivos históricos sin cargar simultáneamente todo su contenido en memoria, pero el procesamiento continúa realizándose en un único entorno local. Por este motivo, el trabajo no evalúa capacidades de procesamiento distribuido, escalabilidad horizontal ni ejecución sobre una infraestructura basada en clústeres.

La selección de los puntos de medida constituye otra limitación del caso de estudio. Se han utilizado 50 puntos seleccionados en función del número de registros identificados como válidos en el archivo de referencia utilizado durante el proceso de selección. Este procedimiento permite delimitar el volumen procesado, pero no corresponde a un muestreo espacial estratificado por distrito, tipo de vía o localización. En consecuencia, los resultados obtenidos describen los puntos incluidos en el análisis y no garantizan la representatividad estadística del conjunto de la red de tráfico de Madrid.

La cobertura temporal de los datos tampoco es uniforme. Aunque el periodo analizado comprende los años 2023 y 2024, determinados periodos presentan un menor número de observaciones disponibles. Esta circunstancia resulta especialmente visible en octubre y noviembre de 2024 dentro del conjunto de prueba. El proceso de preparación conserva únicamente las observaciones que cumplen los criterios establecidos y no completa artificialmente las horas ausentes.

Esta disponibilidad condiciona también la evaluación predictiva. Las métricas se calculan sobre las observaciones para las que es posible construir todas las variables requeridas por el modelo a partir de información histórica anterior. En consecuencia, el rendimiento obtenido caracteriza las 371.128 observaciones elegibles del conjunto de prueba de 2024 y no permite

determinar el comportamiento del modelo durante periodos para los que no se dispone de la información previa necesaria.

Otra limitación corresponde a las variables utilizadas para la predicción. El modelo incorpora información temporal, retardos de intensidad, medias móviles y variables operativas retardadas, pero no incluye información externa relacionada con meteorología, festivos, accidentes, obras, eventos u otras incidencias. La posible aportación de estas variables no se ha evaluado en este trabajo y, por tanto, no puede determinarse a partir de los resultados obtenidos.

El alcance de la comparación de modelos también es limitado. RandomForestRegressor se ha evaluado frente a dos estrategias temporales de referencia, basadas respectivamente en la intensidad de la hora anterior y en la correspondiente a 24 horas antes. Los resultados permiten concluir que Random Forest obtiene mejores métricas que estos dos modelos de referencia sobre el conjunto de prueba utilizado, pero no permiten afirmar que sea el algoritmo con mejor rendimiento entre todas las alternativas posibles.

El planteamiento predictivo presupone además la disponibilidad de información histórica reciente en el momento de generar cada predicción. Esta condición puede reproducirse sobre los datos históricos utilizados en la evaluación, pero el proyecto no incluye el despliegue de un servicio de adquisición y predicción en tiempo real. Por tanto, la solución evalúa un escenario predictivo sobre información histórica y no un sistema de predicción desplegado en un entorno de producción.

Finalmente, la capa de visualización se ha desarrollado mediante Power BI Desktop y utiliza los archivos generados previamente por el pipeline. No se ha realizado un despliegue en Power BI Service ni se ha implementado un mecanismo de actualización automática conectado a las fuentes originales. El informe constituye, por tanto, una capa interactiva de exploración de los datos históricos y de los resultados obtenidos, y no un sistema actualizado de forma continua.

8.2.Trabajo futuro

A partir de las limitaciones identificadas, una primera línea de trabajo consiste en ampliar la cobertura temporal y espacial del caso de estudio. El procesamiento de un periodo más

extenso y de un mayor número de puntos de medida permitiría evaluar el comportamiento de la arquitectura con un conjunto más amplio de información. Esta ampliación podría complementarse con un procedimiento de selección que considere la distribución espacial de los sensores o, si los recursos disponibles lo permiten, con el procesamiento del conjunto completo de puntos de medida.

Una segunda línea consiste en enriquecer el conjunto analítico mediante variables externas. Entre las posibles fuentes se encuentran información meteorológica, calendarios de días festivos, accidentes, obras, eventos u otras incidencias relacionadas con la movilidad. La incorporación de estas variables requeriría analizar su disponibilidad temporal, integrarlas en el pipeline y evaluar experimentalmente si mejoran la capacidad predictiva respecto al modelo actual.

También resulta de interés ampliar la comparación de algoritmos predictivos. El trabajo realizado se ha centrado en Random Forest y dos modelos temporales de referencia, por lo que podrían evaluarse otros métodos de regresión y series temporales. Para que los resultados fueran comparables, esta evaluación debería mantener las mismas observaciones, particiones temporales y métricas utilizadas en el experimento actual.

Desde el punto de vista de la infraestructura, una ampliación relevante sería evaluar la ejecución del procesamiento mediante tecnologías distribuidas. Esta evolución permitiría estudiar experimentalmente el comportamiento de la solución al incrementar el volumen de datos y comparar sus prestaciones con las obtenidas mediante el procesamiento local. Entre las alternativas existentes, Apache Spark proporciona un motor de procesamiento distribuido y Azure Databricks permite utilizarlo dentro de una plataforma orientada a cargas de ingeniería de datos y análisis. Las arquitecturas documentadas para este entorno contemplan tanto fuentes procesadas por lotes como flujos de datos continuos.

En este contexto también podría estudiarse una organización de los datos mediante capas diferenciadas según su grado de procesamiento. La denominada arquitectura medallón distingue habitualmente una capa Bronze para los datos originales, una capa Silver para información validada y preparada y una capa Gold destinada a conjuntos agregados o preparados para consumo analítico. Esta organización constituye un patrón recomendado en entornos lakehouse y tendría que evaluarse específicamente antes de incorporarlo a la arquitectura desarrollada en este trabajo.

Otra posible evolución consiste en automatizar la adquisición y el procesamiento de nuevos datos. Esto requeriría sustituir el flujo actual basado en archivos históricos por mecanismos capaces de detectar o recibir nuevas observaciones, ejecutar las transformaciones necesarias y actualizar posteriormente los conjuntos utilizados por el modelo y la visualización. La incorporación de procesamiento continuo tendría sentido únicamente si la frecuencia y disponibilidad de las fuentes justificasen este cambio de arquitectura.

En la capa de visualización podría evaluarse la publicación del informe mediante Power BI Service. Esto permitiría estudiar mecanismos de acceso compartido y, siempre que la configuración y los orígenes de datos fueran compatibles, establecer procesos de actualización del modelo semántico. Power BI Service permite configurar actualizaciones programadas y consultar su historial de ejecución.

Como ampliación adicional, podrían estudiarse mecanismos de notificación asociados a determinados indicadores. Power BI Service permite configurar alertas sobre tarjetas, KPI y medidores incorporados como elementos de un dashboard, que se evalúan cuando se actualizan los datos. Su aplicación al proyecto requeriría definir previamente qué indicadores y umbrales resultan relevantes para el escenario de movilidad analizado.

Estas líneas permitirían estudiar la evolución del prototipo hacia una solución con mayor cobertura, automatización y capacidad de procesamiento. No obstante, cada una requeriría una nueva fase de implementación y evaluación antes de poder considerarse una capacidad demostrada de la arquitectura.

8.3. Prospectiva de la arquitectura

A medio plazo, la arquitectura desarrollada proporciona una base sobre la que evaluar nuevas fuentes de datos, mecanismos de procesamiento y estrategias predictivas. La separación existente entre adquisición, preparación, modelado, evaluación y visualización permite identificar los componentes que tendrían que modificarse ante cambios en los requisitos o en las características de los datos.

Una posible evolución consistiría en mantener esta separación funcional y sustituir progresivamente determinados componentes locales por mecanismos de almacenamiento y procesamiento adecuados a mayores volúmenes o frecuencias de actualización. Esta

transición permitiría comprobar experimentalmente hasta qué punto las decisiones adoptadas en el prototipo continúan siendo válidas al aumentar la cantidad de información procesada o introducir mecanismos de actualización más frecuentes.

La organización de los datos podría evolucionar igualmente hacia una separación más formal entre información original, información validada y conjuntos preparados para análisis y visualización. Este planteamiento presenta similitudes conceptuales con arquitecturas por capas utilizadas en entornos lakehouse, aunque su adopción requeriría redefinir los mecanismos de almacenamiento, procesamiento y control de calidad empleados actualmente.

La posible aplicación de la arquitectura a otros dominios constituye otra línea de prospectiva. La organización modular podría utilizarse como punto de partida para estudiar conjuntos de datos temporales pertenecientes a otros ámbitos. Sin embargo, cada escenario requeriría redefinir las fuentes de información, las reglas de calidad, la variable objetivo, las variables predictoras, los criterios de evaluación y la capa de visualización. Por este motivo, la reutilización en otros dominios se plantea como una posibilidad de diseño y no como una capacidad validada por los resultados del presente trabajo.

En conjunto, la evolución de la arquitectura dependerá de las necesidades concretas del escenario en el que se aplique. El trabajo realizado demuestra su funcionamiento sobre datos históricos de tráfico de Madrid, con procesamiento local por lotes y una capa de visualización interactiva. La ampliación hacia mayores volúmenes de información, procesamiento distribuido, actualización continua o nuevos dominios requerirá nuevas implementaciones y evaluaciones específicas.

Referencias bibliográficas

- ChatGPT*. (s/f). ChatGPT. Recuperado el 13 de septiembre de 2026, de <https://chatgpt.com/>
- Kitchin, R. (2014). Big Data, new epistemologies and paradigm shifts. *Big Data & Society*, 1(1). <https://doi.org/10.1177/2053951714528481>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/a:1010933404324>
- Tráfico. Histórico de datos del tráfico desde 2013 - Portal de datos abiertos del Ayuntamiento de Madrid. (s. f.). <https://datos.madrid.es/dataset/208627-0-transporte-ptomedida-historico>
- Tráfico. Ubicación de los puntos de medida del tráfico - Portal de datos abiertos del Ayuntamiento de Madrid. (s. f.-b). <https://datos.madrid.es/dataset/202468-0-intensidad-traffic>
- Jonker, T. K. A. (2026, 28 mayo). ¿Qué es la arquitectura de datos? IBM. <https://www.ibm.com/es-es/think/topics/data-architecture>
- Kosinski, A. B. M. (2026, 27 mayo). ¿Qué es el big data? IBM. <https://www.ibm.com/es-es/think/topics/big-data>
- Vibhareddyv. (s. f.). Arquitecturas de macrodatos - Azure Architecture Center. Microsoft Learn. <https://learn.microsoft.com/es-es/azure/architecture/databases/guide/big-data-architectures>
- Kosinski, A. J. J. H. M. (2026, 27 mayo). Qué es un almacén de datos. <https://www.ibm.com/es-es/think/topics/data-warehouse>
- Kosinski, M. (2025, 28 noviembre). Almacenes de datos vs. data lakes vs. lakehouses de datos. <https://www.ibm.com/es-es/think/topics/data-warehouse-vs-data-lake-vs-data-lakehouse>
- Breiman, L. (2001b). Random forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/a:1010933404324>
- Common pitfalls and recommended practices. (s. f.). Scikit-learn. https://scikit-learn.org/stable/common_pitfalls.html
- TimeSeriesSplit. (s. f.). Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

JulCsc. (s. f.). Informes en el servicio Power BI - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/explore-reports/end-user-reports>

JulCsc. (s. f.-b). Personalización del panel Visualizaciones en Power BI Desktop y el servicio Power BI - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/visuals/power-bi-report-visualizations>

JulCsc. (s. f.-b). Introducción a Power BI Desktop - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/fundamentals/desktop-getting-started>

JulCsc. (s. f.-b). Introducción a los paneles para los diseñadores de Power BI - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/create-reports/service-dashboards>

pandas documentation — pandas 3.0.5 documentation. (s. f.). <https://pandas.pydata.org/docs/>

Haya, P. (2022, 5 enero). La metodología CRISP-DM en ciencia de datos. Instituto de Ingeniería del Conocimiento. <https://www.iic.uam.es/innovacion/metodologia-crisp-dm-ciencia-de-datos/>

BOE-A-2007-19814 Ley 37/2007, de 16 de noviembre, sobre reutilización de la información del sector público. (s. f.-b). <https://www.boe.es/buscar/act.php?id=BOE-A-2007-19814>

Condiciones de uso - Portal de datos abiertos del Ayuntamiento de Madrid. (s. f.). <https://datos.madrid.es/pages/condiciones-de-uso>

BOE-A-2025-24545 Ley 9/2025, de 3 de diciembre, de Movilidad Sostenible. (s. f.). <https://www.boe.es/buscar/act.php?id=BOE-A-2025-24545>

David, J. (2025, 14 julio). Top 10 clean code practices to reduce technical debt. Sparity. <https://www.sparity.com/blogs/the-importance-of-business-intelligence/>

Amazon Web Services. (s. f.). ¿Qué es extracción, transformación y carga (ETL)? Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/etl/>

Ibm. (2026, 2 junio). Calidad de los datos. IBM. <https://www.ibm.com/es-es/think/topics/data-quality>

Capdevila, F. (2026, 22 abril). ► ¿Qué es Power BI? Guía completa: para qué sirve, cómo funciona y precios (2026). Hako IT. <https://www.hakoit.com/que-es-power-bi/>

Cerveró, E. (2026, 21 enero). ¿Qué es Power BI, para qué sirve y cómo funciona? Aglaia. <https://aglaia.es/blog/power-bi/que-es-power-bi-para-que-sirve-y-como-funciona/>

Estudio de la movilidad con Big Data | Ministerio de Transportes y Movilidad Sostenible. (s. f.). <https://www.transportes.gob.es/ministerio/proyectos-singulares/estudio-de-movilidad-con-big-data>

LA REVOLUCIÓN DEL BIG DATA EN EL TRANSPORTE: TRANSFORMANDO LA MOVILIDAD EN LA ERA DIGITAL. (s. f.). Master Big Data y Data Science UNED. <https://www.masterbigdataonline.com/blog/20250109-big-data-sector-transportes.html>

Martínez, S. (2026, 30 enero). Ciclo de vida de los datos: qué es, etapas y cómo gestionarlo correctamente. Inesdi. <https://www.inesdi.com/blog/ciclo-vida-datos/>

JulCsc. (s. f.). Introducción a Power BI Desktop - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/fundamentals/desktop-getting-started>

Ciclo de vida de los datos - AWS Orientación prescriptiva. (s. f.). https://docs.aws.amazon.com/es_es/prescriptive-guidance/latest/modern-data-centric-use-cases/data-lifecycle.html

Hai, R., Koutras, C., Quix, C., & Jarke, M. (2023). Data Lakes: A Survey of Functions and Systems. IEEE Transactions On Knowledge And Data Engineering, 35(12), 12571-12590. <https://doi.org/10.1109/tkde.2023.3270101>

Bienvenido - EMT data. (s. f.). <https://datos.emtmadrid.es/>

Levin, W. L. C. D. B. O. (2020, 7 enero). NIST Big Data Interoperability Framework: Volume 6, Reference Architecture. NIST. <https://www.nist.gov/publications/nist-big-data-interoperability-framework-volume-6-reference-architecture>

JulCsc. (s. f.-b). Power BI documentation - Power BI. Microsoft Learn. <https://learn.microsoft.com/en-us/power-bi/>

BOE-A-2018-16673 Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. (s. f.). <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>

Reglamento - 2016/679 - ES - GDPR - EUR-LEX. (s. f.). <https://eur-lex.europa.eu/eli/reg/2016/679/oj?locale=es>

BOE-A-2007-19814 Ley 37/2007, de 16 de noviembre, sobre reutilización de la información del sector público. (s. f.-c). <https://www.boe.es/buscar/act.php?id=BOE-A-2007-19814&>

BOE.es - buscar. (s. f.). <https://www.boe.es/buscar/>

CIDR proceedings. (s. f.). <https://vldb.org/cidrdb/2021/>

Spark Structured Streaming | Apache Spark. (s. f.). <https://spark.apache.org/streaming/>

Anexo A. Privacidad y protección de datos

Este anexo analiza la naturaleza de los datos utilizados en el Trabajo Fin de Máster y su relación con la normativa de protección de datos personales. El análisis se realiza sobre los conjuntos de datos empleados realmente durante el desarrollo y la validación de la solución.

El caso práctico desarrollado utiliza datos abiertos de tráfico urbano publicados por el Ayuntamiento de Madrid. En concreto, se emplearon los datos históricos correspondientes a los años 2023 y 2024 y la información de ubicación de los puntos de medida. Estos conjuntos se utilizaron para implementar y validar las etapas de ingesta, limpieza, transformación, modelado predictivo y visualización interactiva de la arquitectura desarrollada. El Portal de Datos Abiertos del Ayuntamiento de Madrid identifica ambos conjuntos como información pública relacionada con los puntos de medida de tráfico de la ciudad.

A.1. Tipo de datos tratados

Los datos históricos utilizados contienen información asociada a los sensores de tráfico, entre la que se encuentran el identificador del punto de medida, la fecha y hora de la observación, la intensidad de circulación, la ocupación, la carga, la velocidad media y variables técnicas relacionadas con la medición. Estos datos se complementaron con la información de ubicación de los puntos de medida, que incluye variables como el identificador del sensor, el distrito, su denominación y sus coordenadas geográficas.

El identificador empleado en estos conjuntos corresponde a un punto de medida de la infraestructura de tráfico y no a una persona física. Durante el desarrollo del trabajo no se utilizaron nombres, documentos de identidad, matrículas de vehículos, identificadores de dispositivos móviles, direcciones de personas, trayectorias individuales ni otras variables destinadas a identificar o realizar un seguimiento de personas físicas.

El Reglamento (UE) 2016/679 y la Ley Orgánica 3/2018 regulan el tratamiento de información relativa a personas físicas identificadas o identificables. La Ley Orgánica 3/2018 establece su aplicación a los tratamientos automatizados o no automatizados de datos personales en los términos previstos por la normativa de protección de datos. Los conjuntos utilizados en este trabajo describen mediciones realizadas por puntos de control de tráfico y la ubicación de

dichos dispositivos, sin incorporar identificadores personales en las variables utilizadas para el análisis.

A.2. Aplicación de la normativa de protección de datos

A partir de la revisión de las variables utilizadas en el proyecto, no se ha identificado tratamiento de información relativa a personas físicas identificadas o identificables. El objeto del análisis son los puntos de medida y las condiciones agregadas del tráfico registradas en distintos momentos temporales.

Por este motivo, el tratamiento realizado en el caso práctico no se ha basado en información personal de conductores, peatones u otros usuarios de la vía. Tampoco se han combinado los datos con otras fuentes con el propósito de identificar personas o reconstruir desplazamientos individuales.

Esta consideración se limita a los conjuntos de datos y variables efectivamente utilizados en el TFM y no implica que cualquier información relacionada con movilidad urbana quede automáticamente fuera del ámbito de la normativa de protección de datos. La determinación depende de la naturaleza de la información utilizada y de la posibilidad de relacionarla con una persona física identificada o identificable.

A.3. Finalidad del tratamiento de datos

Los datos se utilizaron exclusivamente para el desarrollo académico del Trabajo Fin de Máster. Su tratamiento permitió implementar y comprobar el funcionamiento de la arquitectura propuesta, incluyendo la ingesta y preparación de los datos, su agregación temporal, la construcción del conjunto analítico, el entrenamiento y evaluación del modelo predictivo y la representación de los resultados mediante un informe interactivo en Power BI.

El análisis realizado se orientó al comportamiento del tráfico registrado por los puntos de medida seleccionados. Los datos no se utilizaron para adoptar decisiones automatizadas sobre personas, elaborar perfiles individuales, realizar seguimiento de usuarios ni identificar patrones de comportamiento correspondientes a personas concretas.

A.4. Medidas aplicadas durante el desarrollo

Durante el desarrollo del proyecto se mantuvieron separados los datos originales de los conjuntos generados durante las distintas fases de procesamiento. Los archivos originales se conservaron como fuente de entrada, mientras que los resultados de las operaciones de limpieza, agregación, modelado y exportación se almacenaron en directorios diferenciados.

El procesamiento se limitó a las variables necesarias para los objetivos técnicos y analíticos del proyecto. La solución utilizó identificadores de puntos de medida y variables de tráfico, temporales y geográficas necesarias para construir el conjunto analítico y las visualizaciones. No se incorporaron variables destinadas a identificar personas físicas.

También se documentó el origen institucional de los datos y se mantuvo la trazabilidad entre los datos originales y los conjuntos procesados generados por los scripts del proyecto. Esta separación permitió conservar los archivos de origen sin modificarlos y generar de forma independiente los conjuntos empleados posteriormente en el análisis predictivo y en Power BI.

A.5. Reutilización de datos abiertos

Los conjuntos de datos utilizados proceden del Portal de Datos Abiertos del Ayuntamiento de Madrid. El conjunto denominado Tráfico. Histórico de datos del tráfico desde 2013 proporciona los datos históricos registrados por los puntos de medida y se actualiza mensualmente. El conjunto Tráfico. Ubicación de los puntos de medida del tráfico proporciona información asociada a la localización de dichos puntos. El portal identifica estos recursos como conjuntos de datos abiertos y actualmente los publica bajo licencia Creative Commons Attribution 4.0 International.

La reutilización de información del sector público se encuentra regulada en España por la Ley 37/2007, de 16 de noviembre, sobre reutilización de la información del sector público. Entre las condiciones generales que pueden establecerse para la reutilización se encuentran no alterar el contenido de la información, no desnaturalizar su sentido, citar la fuente y mencionar la fecha de la última actualización. La normativa también contempla que, cuando

se proporcione información disociada que pudiera permitir identificar a personas mediante su combinación con otras fuentes, no se revierta dicho procedimiento de disociación.

En el TFM se identifica al Ayuntamiento de Madrid como fuente de los conjuntos utilizados y los datos se emplean exclusivamente dentro del alcance académico y técnico descrito en la memoria.

A.6. Conclusión

Los datos utilizados para desarrollar y validar la solución corresponden a mediciones de tráfico urbano y a información geográfica y descriptiva de los puntos de medida publicados por el Ayuntamiento de Madrid. Las variables empleadas en el proyecto no contienen nombres, documentos de identidad, matrículas, identificadores de dispositivos ni otros elementos utilizados para identificar a personas físicas.

El tratamiento realizado se centró en el análisis agregado del tráfico y en la evaluación de una arquitectura de procesamiento, modelado predictivo y visualización. No se realizaron perfiles individuales, seguimiento de personas ni reconstrucción de trayectorias personales.

Asimismo, se mantuvo la separación entre datos originales y procesados, se documentó el origen de los conjuntos utilizados y se respetó su carácter de información pública reutilizable. Por tanto, dentro del alcance concreto del caso práctico desarrollado, no se ha realizado un tratamiento de datos personales identificados o identificables.

Anexo B. Script de ingesta y limpieza

```
from pathlib import Path
import pandas as pd
import numpy as np

# =====
# CONFIGURACIÓN GENERAL
# =====

BASE = Path(__file__).resolve().parent.parent
RUTA_HISTORICO = BASE / "datos" / "originales" / "historico_trafico"
RUTA_PUNTOS = BASE / "datos" / "originales" / "puntos_medida"
RUTA_PROCESADOS = BASE / "datos" / "procesados" / "definitivos"
RUTA_PROCESADOS.mkdir(parents=True, exist_ok=True)

COLUMNAS_HISTORICO = [
    "id",
    "fecha",
    "tipo_elem",
    "intensidad",
    "ocupacion",
    "carga",
    "vmed",
    "error",
    "periodo_integracion"
]

CHUNKSIZE = 500_000
MAX_PUNTOS = 50
MIN_REGISTROS_HORA = 4

# =====
# FUNCIONES AUXILIARES
# =====

#Función que lee el Excel de coordenadas y prepara la información geográfica de los sensores
def leer_puntos_medida():
    """
    Lee el archivo Excel de puntos de medida y devuelve una tabla limpia
    con id, nombre, distrito, longitud y latitud.
    """
    archivos = sorted(RUTA_PUNTOS.glob("*.xlsx")) + sorted(RUTA_PUNTOS.glob("*.xls")) #se
    ordenan los archivos

    if not archivos:
        raise FileNotFoundError(f"No se ha encontrado ningún archivo Excel en {RUTA_PUNTOS}")
```

```

archivo = archivos[0]
print(f"Leyendo puntos de medida: {archivo.name}")
puntos = pd.read_excel(archivo)

puntos.columns = (
    puntos.columns
    .str.strip() #espacios en blanco
    .str.lower() #minúsculas
    .str.replace(" ", "_") #espacios por guiones
)

columnas_necesarias = ["id", "tipo_elem", "distrito", "nombre", "longitud", "latitud"]
columnas_existentes = [c for c in columnas_necesarias if c in puntos.columns]
puntos = puntos[columnas_existentes].copy() #copia las columnas existentes

puntos["id"] = pd.to_numeric(puntos["id"], errors="coerce") #convierte id a número
puntos = puntos.dropna(subset=["id"]) #elimina filas sin id
puntos = puntos.drop_duplicates(subset=["id"]) #elimina duplicados del sensor
puntos["id"] = puntos["id"].astype(int) #convierte id a entero

if "longitud" in puntos.columns: #si existe longitud en las columnas, convierte a número
    puntos["longitud"] = pd.to_numeric(puntos["longitud"], errors="coerce")

if "latitud" in puntos.columns: #si existe latitud en las columnas, convierte a número
    puntos["latitud"] = pd.to_numeric(puntos["latitud"], errors="coerce")

puntos.to_csv(RUTA_PROCESADOS / "puntos_medida_limpios.csv", index=False, sep=";")
#guarda la tabla limpia

print(f"Puntos de medida limpios: {len(puntos)}")
return puntos

#Función que escoge los 50 sensores con más registros marcados como válidos
def seleccionar_top_puntos(archivo_referencia):
    """
    Selecciona los puntos con más registros válidos en el primer archivo.
    """
    print(f"Seleccionando los {MAX_PUNTOS} puntos con más registros usando:
    {archivo_referencia.name}")

    conteos = {} #guarda los registros válidos de cada sensor

    for chunk in pd.read_csv( #lee el CSV por bloques de 500.000 filas
        archivo_referencia,
        sep=";",
        encoding="utf-8",

```

```

usecols=["id", "error"], #limita la lectura a id y error
chunksize=CHUNKSIZE
):
    chunk = chunk[chunk["error"] #filtra registros cuyo indicador de error es N
    .astype(str) #convierte valores a texto
    .str.strip() #elimina espacios
    .str.upper() == "N"] #mayúsculas
    vc = pd.to_numeric(chunk["id"], errors="coerce").dropna().astype(int).value_counts()

    for id_punto, n in vc.items(): #recorre los conteos del bloque
        conteos[id_punto] = conteos.get(id_punto, 0) + int(n) #recupera el acumulado anterior
        y suma los registros del bloque actual

    serie = pd.Series(conteos, dtype="int64").sort_values(ascending=False) #ordena de mayor
    a menor
    top_ids = serie.head(MAX_PUNTOS).index.astype(int).tolist() #selecciona los 50 primeros

    print(f"Puntos seleccionados: {len(top_ids)}")
    print(f"Primeros puntos seleccionados: {top_ids[:10]}")

    pd.DataFrame({ #crea y guarda un csv con los sensores seleccionados y sus conteos
        "id": top_ids,
        "registros_archivo_referencia": serie.head(MAX_PUNTOS).values
    }).to_csv(RUTA_PROCESADOS / "puntos_seleccionados_modelo.csv", index=False, sep=";")

    return top_ids

#Función que recibe un csv y devuelve un resumen de calidad
def procesar_archivo_historico(archivo, ids_seleccionados):
    """
    Procesa un CSV mensual por bloques:
    - filtra errores
    - convierte fechas
    - convierte numéricos
    - filtra intensidades negativas
    - agrega por hora y punto de medida
    """
    print(f"\nProcesando archivo: {archivo.name}")

    grupos_archivo = [] #lista que guarda los bloques filtrados
    registros_leidos = 0 #cuenta las filas originales
    registros_validos = 0 #cuenta las filas que superen los filtros

    for chunk in pd.read_csv( #lee el archivo por bloques
        archivo,
        sep=";",
        encoding="utf-8",

```

```

usecols=COLUMNAS_HISTORICO,
chunksize=CHUNKSIZE
):
    registros_leidos += len(chunk) #devuelve el número de filas por bloque y lo suma a lo
    acumulado

    chunk = chunk[chunk["id"].isin(ids_seleccionados)].copy() #Comprueba si el id está entre
    los sensores seleccionados

    chunk = chunk[chunk["error"].astype(str).str.strip().str.upper() == "N"].copy() #Filtra
    registros sin error

    chunk["fecha"] = pd.to_datetime(chunk["fecha"], errors="coerce") #Convierte la fecha a
    tipo datetime

    for col in ["intensidad", "ocupacion", "carga", "vmed", "periodo_integracion"]: #Convierte
    en numéricas las variables
        chunk[col] = pd.to_numeric(chunk[col], errors="coerce")

    chunk = chunk.dropna(subset=["id", "fecha", "intensidad"]) #Elimina filas sin id, fecha e
    intensidad
    chunk = chunk[chunk["intensidad"] >= 0].copy() #Descarta las intensidades negativas

    registros_validos += len(chunk) #Suma al contador los registros que han pasado el filtro

    grupos_archivo.append(chunk[["id", "fecha", "intensidad", "ocupacion",
    "carga", "vmed", "periodo_integracion"]].copy()) #Añade a la lista de bloques los
    necesarios

    if not grupos_archivo:
        return pd.DataFrame(), {"archivo": archivo.name, "registros_leidos": registros_leidos,
        "registros_validos": 0, "registros_horarios": 0} #Devuelve un dataframe vacío si no se ha
        añadido ningún bloque

    datos = pd.concat(grupos_archivo, ignore_index=True) #Une los bloques filtrados

    duplicados = int(datos.duplicated(["id", "fecha"]).sum()) #Elimina duplicados de sensor y
    fecha
    if duplicados: #Si existen duplicados el programa se detiene
        raise ValueError(f"{archivo.name}: {duplicados} fechas duplicadas por sensor; "
        "revisar antes de agregar.")

    datos["fecha_hora"] = datos["fecha"].dt.floor("h") #Redondea cada hora a su comienzo

    grupo = datos.groupby(["id", "fecha_hora"], as_index=False).agg( #Agrupar las filas que
    pertenecen al mismo sensor y misma hora
        intensidad_horaria=("intensidad", "mean"), #Se definen agregaciones

```

```

ocupacion_media=("ocupacion", "mean"),
carga_media=("carga", "mean"),
velocidad_media=("vmed", "mean"),
periodo_integracion_medio=("periodo_integracion", "mean"),
registros_15min=("intensidad", "count")
)

cuartos = datos.groupby(["id", "fecha_hora"])["fecha"].nunique() #Cuenta los timestamps
distintos
if (cuartos > 4).any(): #Sale error si alguna hora tiene más de 4 timestamps distintos
    raise ValueError(f"{archivo.name}: más de cuatro timestamps por hora; revisar
frecuencia.")

incompletas = int((grupo["registros_15min"] != MIN_REGISTROS_HORA).sum()) #Cuenta las
horas que no tienen cuatro registros
grupo = grupo[grupo["registros_15min"] == MIN_REGISTROS_HORA].copy() #Conserva las
horas completas

minutos = datos["fecha"].dt.minute #Comprueba que el minuto está entre 0, 15, 30 y 45
if not minutos.isin([0, 15, 30, 45]).all():
    raise ValueError(f"{archivo.name}: timestamps fuera de los cuartos esperados.")

print(f"Registros leídos: {registros_leidos}")
print(f"Registros válidos tras limpieza: {registros_validos}")
print(f"Registros horarios generados: {len(grupo)}")

resumen = { #Diccionario con las estadísticas del mes
    "archivo": archivo.name,
    "registros_leidos": registros_leidos,
    "registros_validos": registros_validos,
    "registros_horarios": len(grupo),
    "horas_incompletas_excluidas": incompletas
}
return grupo, resumen

# =====
# PROCESO PRINCIPAL
# =====

def main():
    print("=" * 100)
    print("INICIO DEL PROCESO DE INGESTA Y LIMPIEZA")
    print("=" * 100)

    archivos_historico = sorted(RUTA_HISTORICO.glob("*.csv")) #Ordena todos los archivos

    if not archivos_historico: #Si no los encuentra, sale error

```

```

    raise FileNotFoundError(f"No se han encontrado CSV en {RUTA_HISTORICO}")
print(f"Archivos históricos encontrados: {len(archivos_historico)}")

puntos = leer_puntos_medida() #Utiliza el primer csv ordenado
ids_seleccionados = seleccionar_top_puntos(archivos_historico[0])

resultados = []
resumenes = []

for archivo in archivos_historico: #Recorre los archivos procesados y los muestra
    print("Procesando:", archivo.name, flush=True)
    mes, resumen = procesar_archivo_historico(archivo, ids_seleccionados)

    if not mes.empty: #Si no está vacío el dataframe, lo añade a la lista de resultados
        resultados.append(mes)
        resumenes.append(resumen)
        print(resumen, flush=True)

if not resultados: #Si no hay datos, da error
    raise ValueError("No se han generado registros válidos.")

print("\nUniando todos los meses procesados...")

trafico = pd.concat(resultados, ignore_index=True) #Une los dataframes en una tabla

if trafico.duplicated(["id", "fecha_hora"]).any(): #Comprueba que no hayan mismas horas
    en un mismo sensor
    raise ValueError("Existen horas duplicadas entre archivos mensuales.")

trafico["año"] = trafico["fecha_hora"].dt.year #Se crean variables temporales
trafico["mes"] = trafico["fecha_hora"].dt.month
trafico["dia"] = trafico["fecha_hora"].dt.day
trafico["hora"] = trafico["fecha_hora"].dt.hour
trafico["dia_semana"] = trafico["fecha_hora"].dt.dayofweek
trafico["es_fin_semana"] = trafico["dia_semana"].isin([5, 6]).astype(int)

trafico = trafico.merge( #Se unen las coordenadas mediante la unión
    puntos,
    on="id",
    how="left",
    validate="many_to_one"
)

trafico = trafico.sort_values(["id", "fecha_hora"]) #Ordena el dataset por sensor y por fecha

trafico.to_csv(RUTA_PROCESADOS / "trafico_limpio.csv", index=False, sep=";") # Exporta el
dataset limpio

```

```

pd.DataFrame(resumenes).to_csv(RUTA_PROCESADOS / "resumen_limpieza.csv",
index=False, sep=";") #Convierte resúmenes en una tabla

cobertura = trafico.groupby(["id", "año",
"mes"]).size().rename("horas_observadas").reset_index() #Agrupar los datos
cobertura.to_csv(RUTA_PROCESADOS / "cobertura_mensual.csv", index=False, sep=";")
print("Dataset generado:", len(trafico), "filas")
print("Salida:", RUTA_PROCESADOS)

print("\n" + "=" * 100)
print("PROCESO FINALIZADO")
print("=" * 100)

if __name__ == "__main__":
    main()

```

Anexo C. Script de modelado predictivo

```

from pathlib import Path
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# =====
# CONFIGURACIÓN
# =====

BASE = Path(__file__).resolve().parent.parent
RUTA_PROCESADOS = BASE / "datos" / "procesados" / "definitivos"

ARCHIVO_ENTRADA = RUTA_PROCESADOS / "trafico_limpio.csv"
ARCHIVO_MODELO = RUTA_PROCESADOS / "trafico_modelo.csv"
ARCHIVO_PREDICCIONES = RUTA_PROCESADOS / "predicciones.csv"
ARCHIVO_METRICAS = RUTA_PROCESADOS / "metricas_modelo.csv"

print("=" * 100)
print("INICIO DEL MODELO PREDICTIVO")
print("=" * 100)

#Entrenamiento hasta septiembre de 2023
FECHA_VALIDACION = pd.Timestamp("2023-10-01") #Validación de octubre a diciembre de
2023
FECHA_PRUEBA = pd.Timestamp("2024-01-01") #Prueba desde enero de 2024

```

```

VARIABLES_OPERATIVAS = ["ocupacion_media", "carga_media", "velocidad_media",
"registros_15min"] #Variable para predecir el tráfico

# =====
# PREPARACIÓN DE LOS DATOS
# =====

#Función para los datos para que cada variable predictiva respete el tiempo y no se mezclen
sensores
def preparar_datos(df):
    df = df.copy()

    df["fecha_hora"] = pd.to_datetime(df["fecha_hora"], errors="coerce")

    df = df.dropna(subset=["id", "fecha_hora", "intensidad_horaria"]) #Elimina las filas sin id,
fecha e intensidad

    df["id"] = pd.to_numeric(df["id"], errors="coerce") #Convierte a numérico
    df = df.dropna(subset=["id"]) #Elimina los que no son válidos
    df["id"] = df["id"].astype(int) #Los transforma en enteros

    if df.duplicated(["id", "fecha_hora"]).any(): #Comprueba duplicados del mismo sensor a la
misma hora
        raise ValueError("Existen duplicados id-hora. Revisa la ingesta.")

    df = df.sort_values(["id", "fecha_hora"]) #Ordena por sensor y por fecha

    grupos = []
    for id_punto, grupo in df.groupby("id", sort=False): #Recorre los datos sensor por sensor
        grupo = grupo.set_index("fecha_hora")
        calendario = pd.date_range(grupo.index.min(), grupo.index.max(), freq="h") #Genera
todas las horas del sensor
        grupo = grupo.reindex(calendario) #Alinea los datos con el calendario completo
        grupo.index.name = "fecha_hora" #Da nombre al índice
        grupo["id"] = id_punto #Asigna el id del sensor
        grupos.append(grupo.reset_index()) #Añade la fecha convertida en columna a la lista

    df = pd.concat(grupos, ignore_index=True) #Concatena los calendarios de sensores en una
tabla
    df = df.sort_values(["id", "fecha_hora"]).reset_index(drop=True)

    y = df.groupby("id")["intensidad_horaria"] #Crea un objeto agrupado por sensor que tiene
la columna de intensidad

    df["intensidad_lag_1h"] = y.shift(1) #Desplaza los valores una fila abajo de cada sensor
    df["intensidad_lag_24h"] = y.shift(24)

```

```

for ventana in [3, 24]: #Recorre dos tamaños de ventana: 3 y 24 horas
    df[f"media_movil_{ventana}h"] = (
        df.groupby("id")["intensidad_horaria"] #Separa las series por sensor
            .transform(lambda s: s #Define una función que recibe la serie de intensidad de un
sensor
                .shift(1) #Desplaza la serie una hora hacia el pasado
                .rolling(ventana, min_periods=ventana) #Crea ventanas cuyos datos estén disponibles
                .mean()) #Calcula la media de cada ventana
    )

for col in VARIABLES_OPERATIVAS:
    if col in df: #Comprueba si la columna existe en el dataframe
        df[f"{col}_lag_1h"] = df.groupby("id")[col].shift(1) #Crea columnas con una versión con
sufijo de una hora

df["año"] = df["fecha_hora"].dt.year
df["mes"] = df["fecha_hora"].dt.month
df["dia"] = df["fecha_hora"].dt.day
df["hora"] = df["fecha_hora"].dt.hour
df["dia_semana"] = df["fecha_hora"].dt.dayofweek
df["es_fin_semana"] = df["dia_semana"].isin([5, 6]).astype(int) #Convierte la condición de
sábado o domingo en 1 o 0

features = ["id", "mes", "dia", "hora", "dia_semana", "es_fin_semana", #Lista columnas
entrada Random Forest
            "intensidad_lag_1h", "intensidad_lag_24h",
            "media_movil_3h", "media_movil_24h"]

features += [f"{c}_lag_1h" for c in VARIABLES_OPERATIVAS if c in df] #Versiones retardadas
de variables operativas
df_modelo = df.dropna(subset=features + ["intensidad_horaria"]).copy() #Elimina filas
return df_modelo, features, df

# =====
# CÁLCULO DE MÉTRICAS
# =====

#Función para el cálculo de métricas
def calcular_metricas(y_real, y_pred, nombre, conjunto): #Recibe valores
    return {"modelo": nombre, "conjunto": conjunto, #Devuelve un diccionario con las métricas
y el número de observaciones
            "MAE": mean_absolute_error(y_real, y_pred),
            "RMSE": np.sqrt(mean_squared_error(y_real, y_pred)),
            "R2": r2_score(y_real, y_pred), "n": len(y_real)}

def main():
    print("=" * 80)

```

```

print("INICIO DEL MODELO PREDICTIVO")
print("=" * 80)

df = pd.read_csv(ARCHIVO_ENTRADA, sep=";") #Lee el dataset corregido
df_modelo, features, calendario = preparar_datos(df)

print("Registros originales:", len(df))
print("Filas calendario:", len(calendario))
print("Registros disponibles para modelado:", len(df_modelo))
print("Variables:", features)

train = df_modelo[df_modelo["fecha_hora"] < FECHA_VALIDACION].copy() #Selecciona filas
antes al 1 octubre

val = df_modelo[(df_modelo["fecha_hora"] >= FECHA_VALIDACION) & #Filas desde 1
octubre a 31 diciembre
(df_modelo["fecha_hora"] < FECHA_PRUEBA)].copy()

test = df_modelo[df_modelo["fecha_hora"] >= FECHA_PRUEBA].copy() #Filas desde el 1 de
enero 2024

if train.empty or val.empty or test.empty: #Comprueba que ninguna partición esté vacía
    raise ValueError("Una partición temporal está vacía.")

print("Entrenamiento:", len(train), "Validación:", len(val), "Prueba:", len(test))
target = "intensidad_horaria" #Define la columna que quiero predecir

# =====
# CREACIÓN Y ENTRENAMIENTO RANDOM FOREST
# =====

modelo_rf = RandomForestRegressor( #Crea el modelo con los parámetros
    n_estimators=100, max_depth=14, min_samples_leaf=5,
    random_state=42, n_jobs=-1)
modelo_rf.fit(train[features], train[target]) #Entrena el modelo

pred_val = modelo_rf.predict(val[features]) #Aplica el modelo al conjunto de validación
validacion = pd.DataFrame([ #Crea una tabla con las métricas de tres modelos
    calcular_metricas(val[target], val["intensidad_lag_1h"], "Persistencia", "validacion"),
    calcular_metricas(val[target], val["intensidad_lag_24h"], "Baseline lag 24h", "validacion"),
    calcular_metricas(val[target], pred_val, "Random Forest Regressor", "validacion")
])
validacion.to_csv(RUTA_PROCESADOS / "metricas_validacion.csv", index=False, sep=";")

train_final = pd.concat([train, val], ignore_index=True)
modelo_rf.fit(train_final[features], train_final[target]) #Vuelve a entrenar el modelo
pred_rf = modelo_rf.predict(test[features]) #Genera las predicciones sobre 2024

```

```

pred_rf = np.maximum(pred_rf, 0) #Compara cada predicción con 0 y conserva el mayor

metricas = pd.DataFrame([ #Calcula las métricas definitivas de los tres modelos
    calcular_metricas(test[target], test["intensidad_lag_1h"], "Persistencia", "prueba"),
    calcular_metricas(test[target], test["intensidad_lag_24h"], "Baseline lag 24h", "prueba"),
    calcular_metricas(test[target], pred_rf, "Random Forest Regressor", "prueba")
])
metricas.to_csv(ARCHIVO_METRICAS, index=False, sep=";")
print(metricas.to_string(index=False))

predicciones = test.copy()
predicciones["prediccion_persistencia"] = test["intensidad_lag_1h"] #Añade tres columnas
con predicciones
predicciones["prediccion_baseline"] = test["intensidad_lag_24h"]
predicciones["prediccion_rf"] = pred_rf

predicciones["error_rf"] = predicciones[target] - predicciones["prediccion_rf"] #Calcula el
error con signo
predicciones["error_abs_rf"] = predicciones["error_rf"].abs() #Calcula el error absoluto

columnas = ["id", "fecha_hora", "año", "mes", "dia", "hora", "dia_semana", #Valores para
PowerBi
    "es_fin_semana", target, "prediccion_persistencia",
    "prediccion_baseline", "prediccion_rf", "error_rf", "error_abs_rf",
    "latitud", "longitud", "nombre", "distrito"]

predicciones[[c for c in columnas if c in predicciones]].to_csv(
    ARCHIVO_PREDICCIONES, index=False, sep=";")

df_modelo.to_csv(ARCHIVO_MODELO, index=False, sep=";")

pd.DataFrame({"variable": features, "importancia": modelo_rf.feature_importances_}
    ).sort_values("importancia", ascending=False).to_csv(
    RUTA_PROCESADOS / "importancia_variables.csv", index=False, sep=";")

pd.DataFrame([ #Crea un csv con el número de registros de cada partición
    {"conjunto": "entrenamiento", "registros": len(train)},
    {"conjunto": "validacion", "registros": len(val)},
    {"conjunto": "entrenamiento_final", "registros": len(train_final)},
    {"conjunto": "prueba", "registros": len(test)}
]).to_csv(RUTA_PROCESADOS / "resumen_modelo.csv", index=False, sep=";")
print("Resultados guardados en:", RUTA_PROCESADOS)

if __name__ == "__main__":
    main()

```

Anexo D. Script de exportación a Power BI

```
from pathlib import Path
import pandas as pd

# =====
# CONFIGURACIÓN
# =====

BASE = Path(__file__).resolve().parent.parent
RUTA_PROCESADOS = BASE / "datos" / "procesados" / "definitivos"
RUTA_POWERBI = RUTA_PROCESADOS / "powerbi"
RUTA_POWERBI.mkdir(parents=True, exist_ok=True)

# =====
# CARGA DE ARCHIVOS
# =====

print("=" * 100)
print("PREPARANDO ARCHIVOS PARA POWER BI")
print("=" * 100)

def main():
    trafico = pd.read_csv(RUTA_PROCESADOS / "trafico_limpio.csv", sep=";")
    predicciones = pd.read_csv(RUTA_PROCESADOS / "predicciones.csv", sep=";")
    metricas = pd.read_csv(RUTA_PROCESADOS / "metricas_modelo.csv", sep=";")
    importancias = pd.read_csv(RUTA_PROCESADOS / "importancia_variables.csv", sep=";")
    resumen_limpieza = pd.read_csv(RUTA_PROCESADOS / "resumen_limpieza.csv", sep=";")
    puntos = pd.read_csv(RUTA_PROCESADOS / "puntos_medida_limpios.csv", sep=";")
    resumen_modelo = pd.read_csv(RUTA_PROCESADOS / "resumen_modelo.csv", sep=";")

    # =====
    # PREPARACIÓN DE FECHAS
    # =====

    trafico["fecha_hora"] = pd.to_datetime(trafico["fecha_hora"]) #Convierte de texto a
datetime
    predicciones["fecha_hora"] = pd.to_datetime(predicciones["fecha_hora"])

    trafico["fecha"] = trafico["fecha_hora"].dt.date #Crea una columna fecha sin la hora
    predicciones["fecha"] = predicciones["fecha_hora"].dt.date

    # =====
    # TABLA RESUMEN GENERAL
    # =====

    resumen_general = pd.DataFrame([ #Dataframe a partir de una lista de diccionarios
```

```

{"indicador": "Registros dataset limpio", "valor": len(trafico)}, #Cuenta las filas del dataset
{"indicador": "Puntos de medida analizados", "valor": trafico["id"].nunique()}, #Cuenta los
id distintos de sensor
{"indicador": "Fecha inicial", "valor": str(trafico["fecha_hora"].min())}, #Obtiene primer y
última fecha y lo convierte a texto
{"indicador": "Fecha final", "valor": str(trafico["fecha_hora"].max())},
{"indicador": "Registros de entrenamiento 2023",
"valor": resumen_modelo.loc[resumen_modelo["conjunto"] == "entrenamiento_final",
#Crea un filtro que localiza la fila del entrenamiento
"registros"].iloc[0]}, #Extrae el primer valor del resultado
{"indicador": "Registros de prueba 2024", #Igual, pero para el conjunto de prueba 2024
"valor": resumen_modelo.loc[resumen_modelo["conjunto"] == "prueba",
"registros"].iloc[0]}
])

# =====
# TABLA DE CALIDAD DE DATOS
# =====

calidad_datos = pd.DataFrame({ #Crea una tabla que analiza los nulos en cada columna
"columna": trafico.columns,
"nulos": trafico.isna().sum().values,
"porcentaje_nulos": trafico.isna().sum().values / len(trafico) * 100
})

# =====
# TABLA TEMPORAL AGREGADA
# =====

trafico_temporal = trafico.groupby(
["año", "mes", "dia_semana", "hora"], as_index=False
).agg( #Calcula indicadores
intensidad_media=("intensidad_horaria", "mean"),
intensidad_total=("intensidad_horaria", "sum"),
ocupacion_media=("ocupacion_media", "mean"),
velocidad_media=("velocidad_media", "mean"),
registros=("id", "count"))

# =====
# TABLA POR PUNTO DE MEDIDA
# =====

trafico_puntos = trafico.groupby( #Tabla que agrupa los datos de cada sensor
["id", "nombre", "distrito", "latitud", "longitud"], as_index=False
).agg(
intensidad_media=("intensidad_horaria", "mean"),
intensidad_total=("intensidad_horaria", "sum"),

```

```

ocupacion_media=("ocupacion_media", "mean"),
velocidad_media=("velocidad_media", "mean"),
registros=("fecha_hora", "count"))

# =====
# TABLA DE PREDICCIONES REDUCIDA
# =====

columnas_pred = [ #Columnas que se exportan de la tabla de predicciones
    "id", "fecha_hora", "fecha", "año", "mes", "dia", "hora",
    "dia_semana", "es_fin_semana", "intensidad_horaria",
    "prediccion_persistencia", "prediccion_baseline", #Predicción hora anterior, predicción
    mismo hora día anterior
    "prediccion_rf", "error_rf", "error_abs_rf", #Predicción generada por Random Forest
    "latitud", "longitud", "nombre", "distrito"
]

predicciones[[c for c in columnas_pred if c in predicciones]]

# =====
# EXPORTACIÓN
# =====

tablas = {
    "trafico_powerbi.csv": trafico,
    "trafico_temporal_powerbi.csv": trafico_temporal,
    "trafico_puntos_powerbi.csv": trafico_puntos,
    "predicciones_powerbi.csv": predicciones[[c for c in columnas_pred if c in predicciones]],
    "metricas_powerbi.csv": metricas,
    "importancia_variables_powerbi.csv": importancias,
    "resumen_limpieza_powerbi.csv": resumen_limpieza,
    "resumen_general_powerbi.csv": resumen_general,
    "calidad_datos_powerbi.csv": calidad_datos,
    "puntos_medida_powerbi.csv": puntos
}
for nombre, tabla in tablas.items(): #Bucle de exportación
    tabla.to_csv(RUTA_POWERBI / nombre, index=False, sep=";", #Guarda cada dataframe
    como csv
        encoding="utf-8-sig", decimal=",")
    print("Exportaciones generadas en:", RUTA_POWERBI)

if __name__ == "__main__":
    main()

```

Anexo E. Imágenes del Dashboard en Power BI

Página 1 – Portada



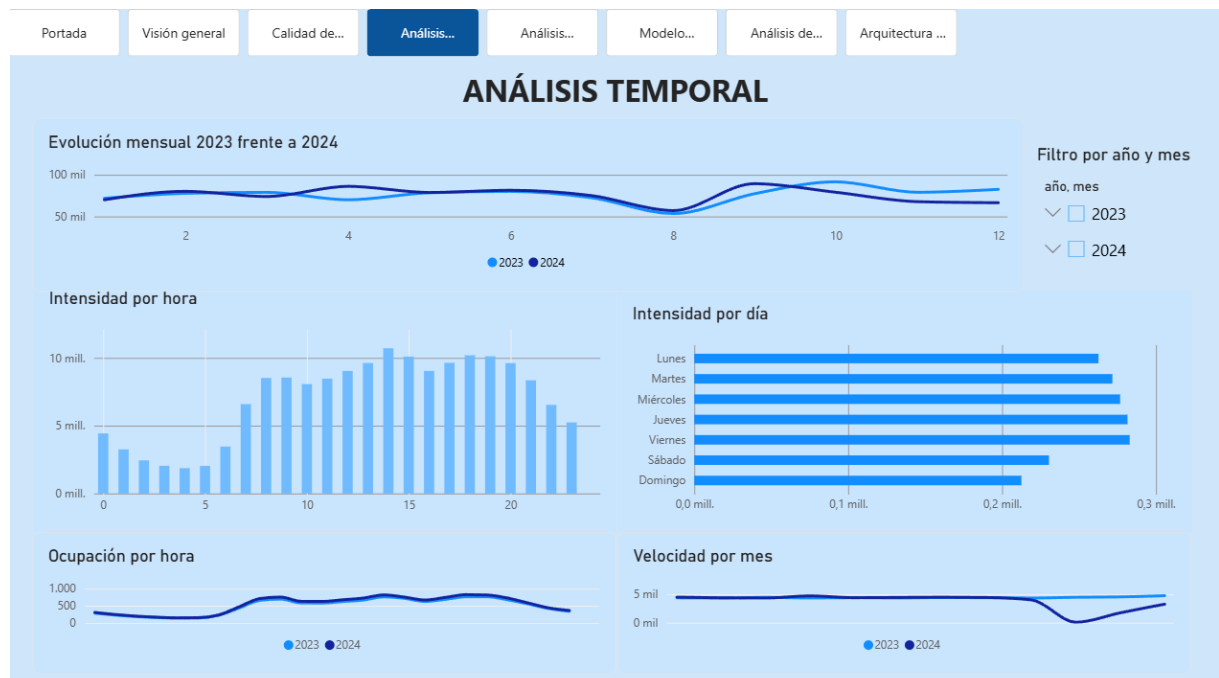
Página 2 – Visión general del tráfico



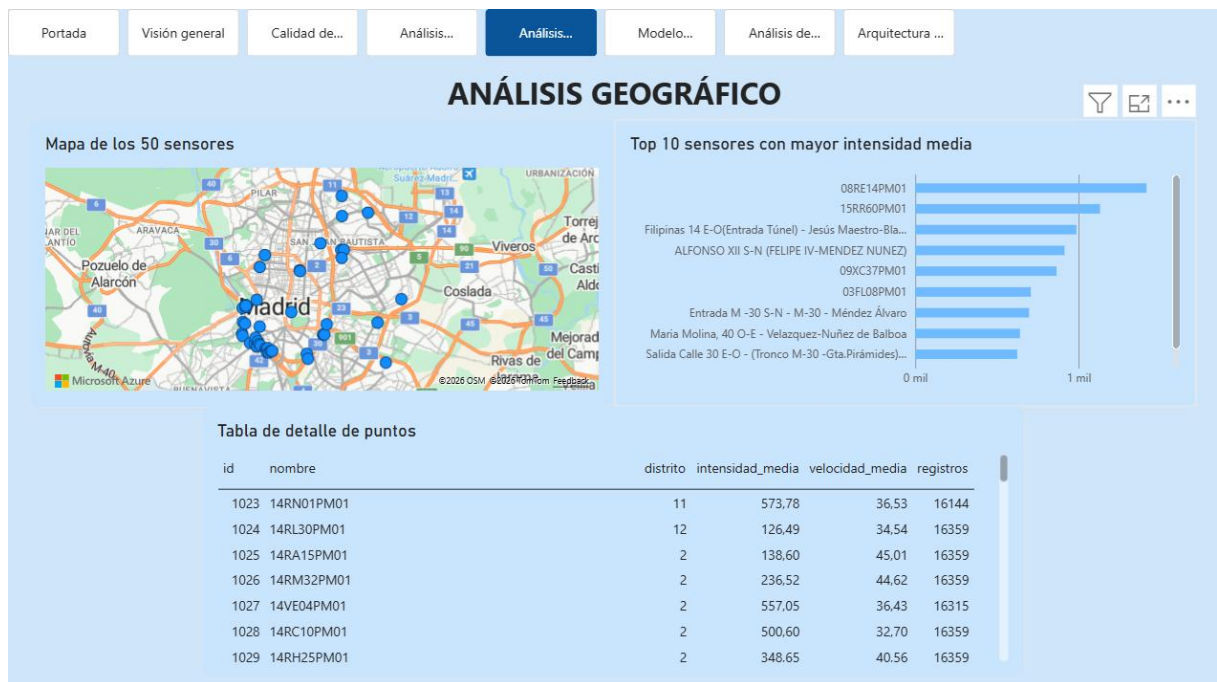
Página 3 – Calidad de los datos



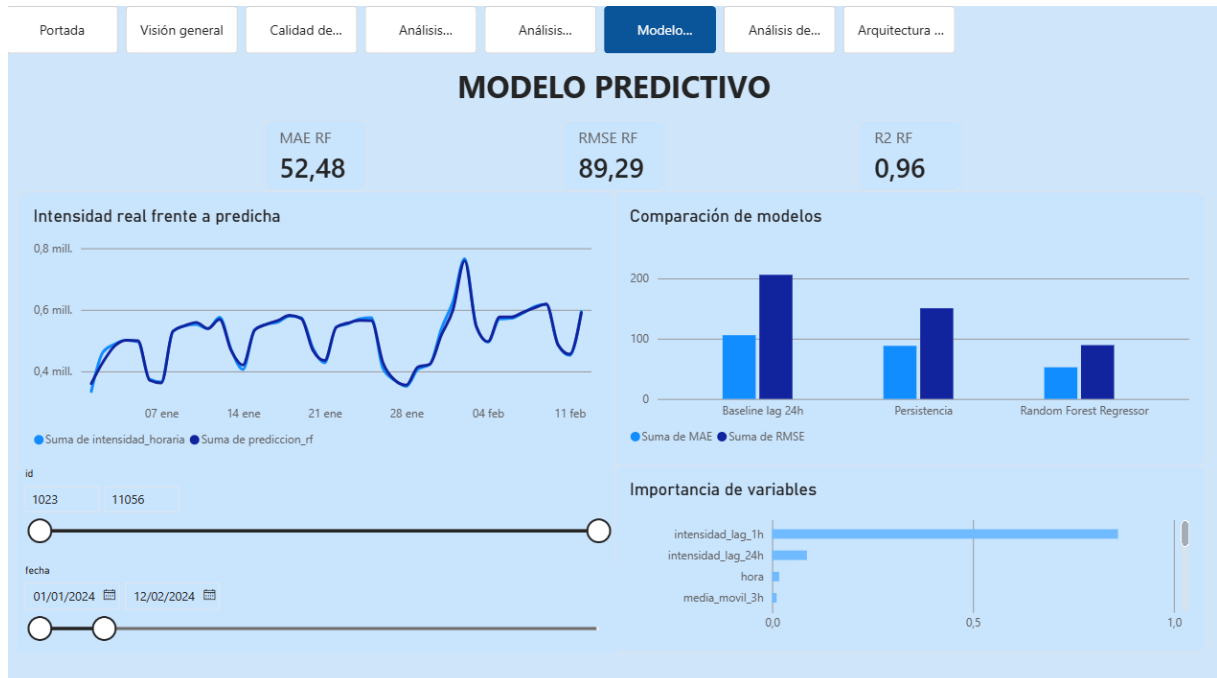
Página 4 – Análisis temporal



Página 5 – Análisis geográfico



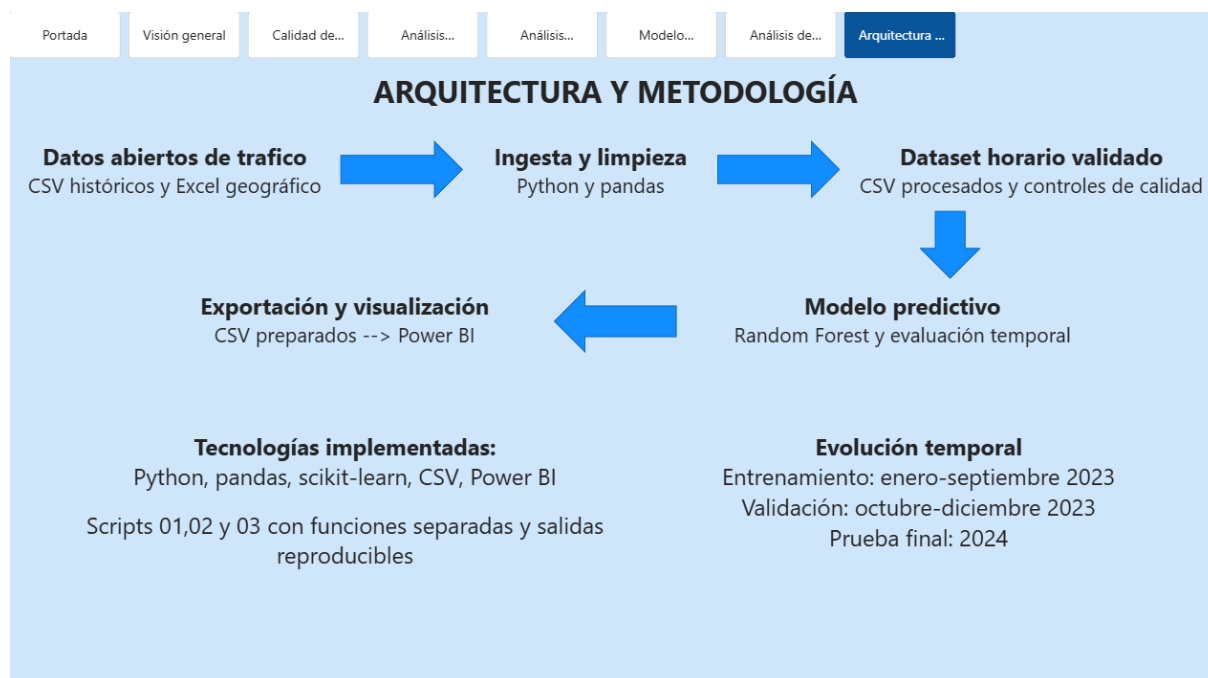
Página 6 – Modelo predictivo



Página 7 – Análisis de errores



Página 8 – Arquitectura y metodología



Anexo F. Declaración de uso de herramientas de IA

Para la elaboración de este Trabajo Fin de Máster se ha utilizado ChatGPT, desarrollado por OpenAI, como herramienta de apoyo en determinadas fases del desarrollo técnico y de la preparación de la memoria.

El uso de la herramienta se ha centrado principalmente en la organización y estructuración de algunos contenidos de la memoria; la revisión y comentario de código Python relacionado con los procesos de ingesta, limpieza, modelado predictivo y exportación de resultados; la detección de posibles errores e incoherencias en el código y en el documento; la revisión formal de textos para mejorar su claridad y organización; y el apoyo en la elaboración de determinados esquemas y recursos gráficos utilizados para explicar la arquitectura y la organización del proyecto.

ChatGPT también se ha utilizado como apoyo durante el proceso de revisión final para localizar contradicciones entre apartados, comprobar la coherencia de las cifras presentadas y proponer posibles mejoras de redacción. Las respuestas proporcionadas por la herramienta no se han considerado por sí mismas una fuente académica o técnica. Las afirmaciones teóricas, normativas y técnicas incorporadas a la memoria se han contrastado con las correspondientes publicaciones académicas, normas, documentación oficial o fuentes institucionales, que se citan de forma independiente en los apartados correspondientes.

En el desarrollo del software, las sugerencias obtenidas mediante ChatGPT fueron revisadas antes de su incorporación al proyecto. La ejecución de los scripts, la comprobación de los archivos generados y la evaluación de los resultados se realizaron sobre el entorno y los datos utilizados en el TFM. En particular, las métricas y resultados presentados en la memoria proceden de la ejecución del pipeline desarrollado y de los archivos generados durante el procesamiento, y no de valores proporcionados por la herramienta de inteligencia artificial.

En los recursos gráficos en los que se utilizó inteligencia artificial como apoyo para su elaboración, esta intervención se identifica de acuerdo con el criterio de atribución utilizado en la memoria. La herramienta no se presenta como autora de los resultados experimentales ni como fuente de las evidencias obtenidas durante la validación.

El uso de ChatGPT se ha realizado, por tanto, como herramienta auxiliar dentro del proceso de elaboración del TFM. La selección de las fuentes empleadas, la comprobación de los

Diseño e implementación de una arquitectura de datos masivos para el análisis predictivo y la visualización interactiva
resultados, la interpretación final de las evidencias y la revisión de la versión entregada de la memoria corresponden a la autora del trabajo.